Apache Kafka工作負載搭配NetApp NFS儲存設備

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/zh-tw/netapp-solutions/data-analytics/kafka-nfs-introduction.html on March 19, 2024. Always check docs.netapp.com for the latest.

目錄

Apache Kafka工作負載搭配NetApp NFS儲存設備 . . . 1
TR-4947：Apache Kafka工作負載搭配NetApp NFS儲存設備-功能驗證與效能 . . . 1
NetApp解決方案可解決NFS對Kafka工作負載的不合理重新命名問題 . . . 2
功能驗證-不太好用的重新命名修正程式 . . . 2
為何選擇適用於Kafka工作負載的NetApp NFS？ . . . 7
AWS的效能總覽與驗證 . . . 18
AWS FSX for NetApp ONTAP 的效能概觀與驗證 . . . 31
透過AFF 內部部署的VMware解決方案實現效能總覽與驗證 . . . 39
結論 . . . 44
何處可找到其他資訊 . . . 45

Apache Kafka工作負載搭配NetApp NFS儲存設備

TR-4947：Apache Kafka工作負載搭配NetApp NFS儲存設備-功能驗證與效能

Shantanu Chakole, Karthikeyan Nagalingam,以及NetApp公司Joe Scott

Kafka是一種分散式發佈訂閱訊息系統、具有強大的佇列、可接受大量的訊息資料。有了Kafka、應用程式就能以非常快速的方式將資料寫入和讀取至主題。由於其容錯能力與擴充性、因此Kafka經常被用在大資料空間、作為快速擷取和移動許多資料串流的可靠方法。使用案例包括串流處理、網站活動追蹤、指標收集與監控、記錄彙總、即時分析等。

雖然正常的Kafka NFS作業運作良好 "不太好重命名" 在NFS上執行的Kafka叢集調整大小或重新分割時、問題會使應用程式當機。這是一個重大問題、因為Kafka叢集必須調整大小或重新分割、才能進行負載平衡或維護。您可以找到其他詳細資料 "請按這裡"。

本文件說明下列主題：

- 這是個不太好重命名的問題、也是解決方案驗證的關鍵
- 降低CPU使用率以縮短I/O等待時間
- 更快的Kafka代理程式恢復時間
- 雲端和內部部署的效能

為什麼要將NFS儲存設備用於Kafka工作負載？

正式作業應用程式中的Kafka工作負載可在應用程式之間串流大量資料。此資料會保留並儲存在Kafka叢集中的Kafka Broker節點中。Kafka也以可用度和平行度而聞名、將主題分成分割區、然後在整個叢集內複寫這些分割區、即可達成此目標。這最終意味著流經Kafka叢集的大量資料通常會增加大小。NFS可在代理程式數量改變時、快速又輕鬆地重新平衡資料。在大型環境中、當代理商數量的變動非常耗時時、會在DAS之間重新平衡資料、而在大多數的Kafka環境中、代理商數量也會經常變動。

其他效益包括：

- 成熟度 NFS是一種成熟的傳輸協定、這表示實作、保護及使用它的大部分層面都已獲得充分的瞭解。
- 開放式 NFS是一種開放式傳輸協定、其持續開發作業記錄於網際網路規格中、以免費開放式網路傳輸協定的形式提供。
- 具成本效益的. NFS是一款低成本的網路檔案共用解決方案、因為它使用現有的網路基礎架構、所以很容易設定。
- 集中管理。 NFS的集中管理可減少個別使用者系統上新增軟體和磁碟空間的需求。
- 分散式 NFS可作為分散式檔案系統、減少對卸除式媒體儲存設備的需求。

為何選擇NetApp來處理Kafka工作負載？

NetApp NFS實作被視為傳輸協定的黃金標準、可用於無數的企業NAS環境。除了NetApp的可信度之外、它還提供下列優點：

- 可靠性與效率
- 擴充性與效能
- 高可用度（HA合作夥伴在NetApp ONTAP 供應叢集中）
- 資料保護
  - *災難恢復（NetApp SnapMirror）。*您的站台當機、或您想要從不同的站台開始、然後從您離開的地方繼續進行。
  - 儲存系統的管理能力（使用NetApp OnCommand 功能進行管理）。
  - *負載平衡。*叢集可讓您從位於不同節點上的資料LIF存取不同的磁碟區。
  - 不中斷營運。 lifs或Volume移動對NFS用戶端而言是透明的。

NetApp解決方案可解決NFS對Kafka工作負載的不合理重新命名問題

Kafka的建置假設基礎檔案系統符合POSIX標準：例如XFS或ext4。在應用程式仍在使用檔案時、Kafka資源重新平衡功能會移除檔案。符合POSIX標準的檔案系統可讓您繼續解除連結。不過、只有在所有檔案參考資料都消失之後、它才會移除檔案。如果基礎檔案系統是網路附加的、NFS用戶端會攔截取消連結的呼叫、並管理工作流程。由於在要取消連結的檔案上有擱置的開啟、NFS用戶端會將重新命名要求傳送至NFS伺服器、並在上次關閉未連結檔案時、對重新命名的檔案執行移除作業。這種行為通常稱為NFS ruted rame、由NFS用戶端協調。

任何使用NFSv3伺服器儲存設備的Kafka代理商、都會因為這種行為而發生問題。不過、NFSv4.x傳輸協定具備多項功能、可讓伺服器負責開啟且未連結的檔案、藉此解決此問題。支援此選用功能的NFS伺服器、會在檔案開啟時將擁有權功能傳達給NFS用戶端。NFS用戶端會在開啟擱置中時停止取消連結管理、並允許伺服器管理流程。雖然NFSv4規格提供實作準則、但直到現在為止、並沒有任何已知的NFS伺服器實作支援此選用功能。

NFS伺服器和NFS用戶端需要進行下列變更、才能解決這個「愚蠢」的重新命名問題：

- * NFS用戶端（Linux）的變更。*檔案開啟時、NFS伺服器會以旗標回應、表示能夠處理已開啟檔案的解除連結。NFS用戶端變更可讓NFS伺服器在旗標出現時處理取消連結。NetApp已針對這些變更更新開放原始碼Linux NFS用戶端。更新後的NFS用戶端現在一般可在RHEL8.7和RHEL9.1中使用。
- * NFS伺服器的變更。* NFS伺服器會持續追蹤開啟狀況。現在、伺服器會管理現有開放式檔案上的取消連結、以符合POSIX語義。關閉最後一個開啟的檔案時、NFS伺服器會啟動檔案的實際移除、進而避免執行不必要的重新命名程序。此功能已在最新版的《S59.12》中實作ONTAP ONTAP 。

上述NFS用戶端和伺服器的變更、讓Kafka能夠安全地享有網路附加NFS儲存設備的所有優點。

功能驗證-不太好用的重新命名修正程式

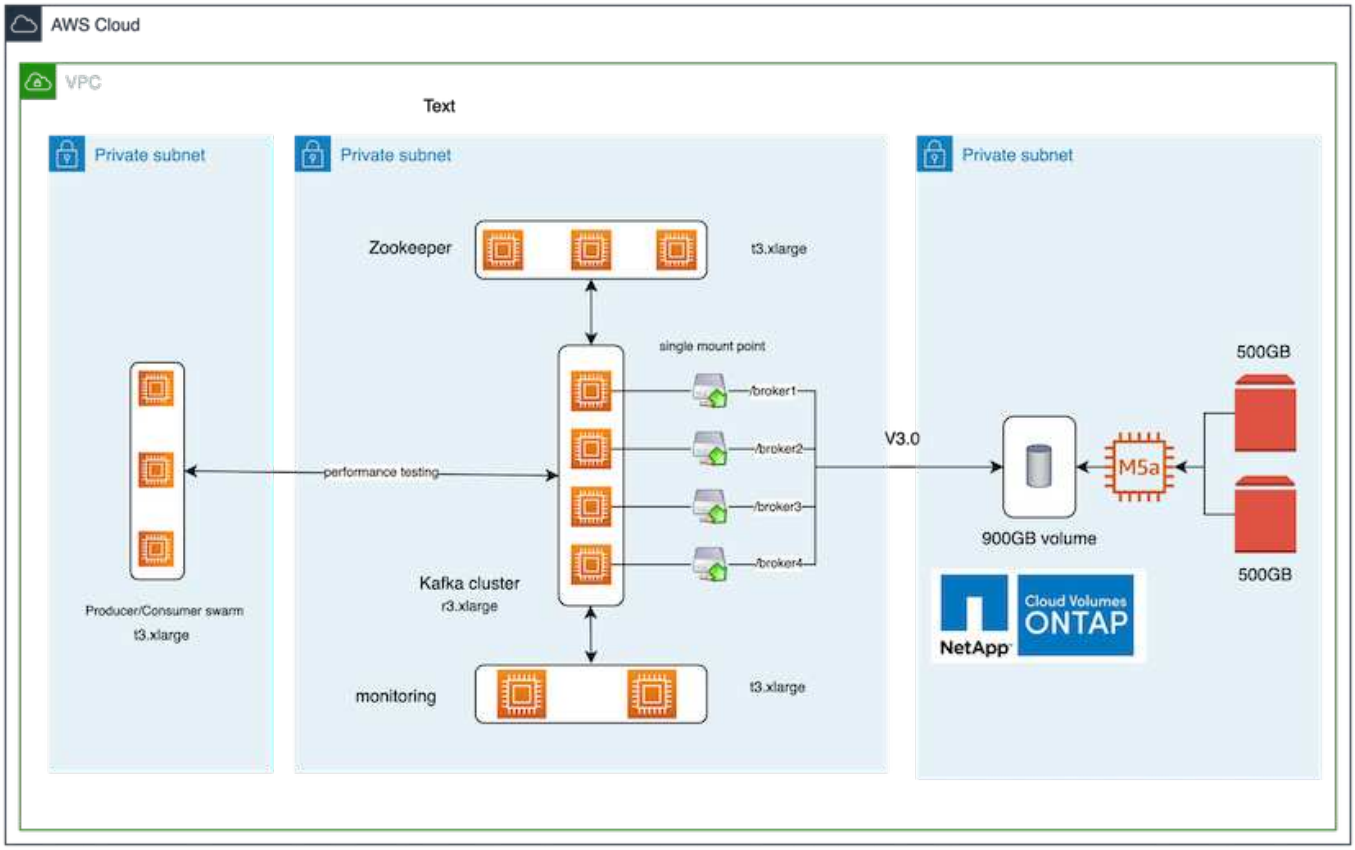
在功能驗證方面、我們發現裝有NFSv3儲存設備掛載的Kafka叢集無法執行像分割區重新分佈等Kafka作業、而裝在NFSv3上的另一個叢集搭配修正程式、則可執行相同的作業、而不會造成任何中斷。

驗證設定

設定會在AWS上執行。下表顯示驗證所使用的不同平台元件和環境組態。

平台元件	環境組態
ConFluent Platform 7.2.1版	• 3個zookeepers–T3.xlarge • 4個代理伺服器–R3.xlarge • 1 x Grafana–T3.xlarge • 1個控制中心–T3.xLarge • 3個製造商/消費者
所有節點上的作業系統	RHEL8.7或更新版本
NetApp Cloud Volumes ONTAP 執行個體	單節點執行個體–M5.2xLarge

下圖顯示此解決方案的架構組態。



架構流程

- *運算*我們使用四節點的Kafka叢集、其中三節點的zookeeper集合體執行於專屬伺服器上。
- *監控*我們使用兩個節點來搭配Prometheus-Grafana。
- 工作負載。*為了產生工作負載、我們使用了一個獨立的三節點叢集、可產生和使用此Kafka叢集。

- * Storage。*我們使用單節點NetApp Cloud Volumes ONTAP 支援執行個體、並將兩個500GB GP2 AWS-EBS磁碟區附加至執行個體。然後、這些磁碟區會透過LIF以單一NFSv4.1磁碟區的形式、公開給Kafka叢集。

所有伺服器都會選擇Kafka的預設屬性。對zookeeper swarm也是如此。

測試方法

1. 更新 `-is-preserve-unlink-enabled true` 至Kafka磁碟區、如下所示：

```
aws-shantanclastrecall-aws::*> volume create -vserver kafka_svm -volume
kafka_fg_vol01 -aggregate kafka_aggr -size 3500GB -state online -policy
kafka_policy -security-style unix -unix-permissions 0777 -junction-path
/kafka_fg_vol01 -type RW -is-preserve-unlink-enabled true
[Job 32] Job succeeded: Successful
```

2. 建立了兩個類似的Kafka叢集、其差異如下：
   - *叢集1.*執行正式作業功能ONTAP 的後端NFS v4.1伺服器9.12.1版是由NetApp CVO執行個體代管。已在代理商上安裝RHEL 8.7/RHEL 9.1。
   - *叢集2.*後端NFS伺服器是手動建立的通用Linux NFSv3伺服器。
3. 兩個Kafka叢集都建立了示範主題。

   叢集1：

```
[root@ip-172-30-0-160 demo]# kafka-topics --bootstrap-server=172.30.0.160:9092,172.30.0.172:9092,172.30.0.188:90
92,172.30.0.123:9092 --describe --topic __a_demo_topic
Topic: __a_demo_topic   TopicId: 2ty29xfhQLq65HKsUQv-pg PartitionCount: 4       ReplicationFactor: 2    Configs:
 min.insync.replicas=1,segment.bytes=1073741824
        Topic: __a_demo_topic   Partition: 0    Leader: 4       Replicas: 4,1   Isr: 4,1        Offline:
        Topic: __a_demo_topic   Partition: 1    Leader: 2       Replicas: 2,4   Isr: 2,4        Offline:
        Topic: __a_demo_topic   Partition: 2    Leader: 3       Replicas: 3,2   Isr: 3,2        Offline:
        Topic: __a_demo_topic   Partition: 3    Leader: 1       Replicas: 1,3   Isr: 1,3        Offline:
```

   叢集2：

```
[root@ip-172-30-0-198 demo]# kafka-topics --bootstrap-server=172.30.0.198:9092,172.30.0.163:9092,172.30.0.221:90
92,172.30.0.204:9092 --describe --topic __a_demo_topic
Topic: __a_demo_topic   TopicId: AwQpsZTQShyeMIhaquCG3Q PartitionCount: 4       ReplicationFactor: 2    Configs:
 min.insync.replicas=1,segment.bytes=1073741824
        Topic: __a_demo_topic   Partition: 0    Leader: 2       Replicas: 2,3   Isr: 2,3        Offline:
        Topic: __a_demo_topic   Partition: 1    Leader: 3       Replicas: 3,1   Isr: 3,1        Offline:
        Topic: __a_demo_topic   Partition: 2    Leader: 1       Replicas: 1,4   Isr: 1,4        Offline:
        Topic: __a_demo_topic   Partition: 3    Leader: 4       Replicas: 4,2   Isr: 4,2        Offline:
```

4. 資料已載入這兩個叢集的新建立主題。這是使用預設Kafka套件隨附的生產商perf-test工具組來完成：

```
./kafka-producer-perf-test.sh --topic __a_demo_topic --throughput -1
--num-records 3000000 --record-size 1024 --producer-props acks=all
bootstrap.servers=172.30.0.160:9092,172.30.0.172:9092,172.30.0.188:9092,
172.30.0.123:9092
```

5. 已使用Telnet:對每個叢集的Broker 1執行健全狀況檢查。
   - 遠端登入 `172.30.0.160 9092`
   - 遠端登入 `172.30.0.198 9092`

     下一個快照會顯示兩個叢集上的代理程式健全狀況檢查是否成功：

```
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.160 9092
Trying 172.30.0.160...
Connected to 172.30.0.160.
Escape character is '^]'.
^[

Connection closed by foreign host.
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.198 9092
Trying 172.30.0.198...
Connected to 172.30.0.198.
Escape character is '^]'.
^[
```

6. 為了觸發故障情況、導致使用NFSv3儲存磁碟區的Kafka叢集當機、我們在兩個叢集上都開始重新指派磁碟分割。磁碟分割重新指派是使用執行 `kafka-reassign-partitions.sh`。詳細程序如下：
   a. 若要重新指派Kafka叢集中某個主題的分割區、我們會產生建議的重新指派組態Json（這是針對兩個叢集執行）。

```
kafka-reassign-partitions --bootstrap
-server=172.30.0.160:9092,172.30.0.172:9092,172.30.0.188:9092,172.30.
0.123:9092 --broker-list "1,2,3,4" --topics-to-move-json-file
/tmp/topics.json --generate
```

   b. 然後儲存產生的重新指派Json `/tmp/reassignment- file.json`。
   c. 實際的分割區重新指派程序是由下列命令觸發：

```
kafka-reassign-partitions --bootstrap
-server=172.30.0.198:9092,172.30.0.163:9092,172.30.0.221:9092,172.30.
0.204:9092 --reassignment-json-file /tmp/reassignment-file.json
-execute
```

7. 重新指派完成幾分鐘後、代理商的另一項健全狀況檢查顯示、使用NFSv3儲存磁碟區的叢集發生錯誤的重新命名問題、並已當機、而使用NetApp ONTAP NFSv4.1儲存磁碟區的叢集1則可在不中斷營運的情況下繼續進行修正作業。

```
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.160 9092
Trying 172.30.0.160...
Connected to 172.30.0.160.
Escape character is '^]'.
^[

Connection closed by foreign host.
shantanu@shantanc-mac-0 ~ % telnet 172.30.0.198 9092
Trying 172.30.0.198...
telnet: connect to address 172.30.0.198: Connection refused
telnet: Unable to connect to remote host
```

- 叢集1-Broler-1處於作用中狀態。
- Cluster2-Broker 1已死機。

8. 查看Kafka記錄目錄後、很明顯、使用NetApp ONTAP 支援NFSv3的叢集1儲存磁碟區搭配修復程式時、會有乾淨的分割區指派、而使用一般NFSv3儲存設備的叢集2則不會因為錯誤的重新命名問題而導致當機。下圖顯示叢集2的分割區重新平衡、導致NFSv3儲存設備發生不實的重新命名問題。

```
/demo/broker_demo_1/__a_demo_topic-1.b31a8dd60fd443b283ffda2ecca9c2b9-delete:
total 40
drwxr-xr-x.   2 nobody nobody  4096 Sep 19 10:37 .
drwxr-xr-x. 246 nobody nobody 32768 Sep 19 10:36 ..
-rw-r--r--.   1 nobody nobody     5 Sep 19 10:22 .nfs0000000025f9008400000045
-rw-r--r--.   1 nobody nobody     0 Sep 19 10:25 .nfs0000000025f91d6800000048

/demo/broker_demo_1/__a_demo_topic-2:
total 832592
drwxr-xr-x.   2 nobody nobody      4096 Sep 19 10:26 .
drwxr-xr-x. 246 nobody nobody     32768 Sep 19 10:36 ..
-rw-r--r--.   1 nobody nobody         5 Sep 19 10:22 .nfs0000000025f91d5500000046
-rw-r--r--.   1 nobody nobody         0 Sep 19 10:25 .nfs0000000025f91fce00000047
-rw-r--r--.   1 nobody nobody  10485760 Sep 19 10:24 00000000000000000000.index
-rw-r--r--.   1 nobody nobody 848113134 Sep 19 10:24 00000000000000000000.log
-rw-r--r--.   1 nobody nobody  10485756 Sep 19 10:24 00000000000000000000.timeindex
-rw-r--r--.   1 nobody nobody         0 Sep 19 10:16 leader-epoch-checkpoint
-rw-r--r--.   1 nobody nobody        43 Sep 19 10:16 partition.metadata
```

下圖顯示使用NetApp NFSv4.1儲存設備重新平衡叢集1的乾淨分割區。

```
/demo/broker_demo_1/__a_demo_topic-0:
total 710932
drwxr-xr-x.  2 nobody nobody      4096 Sep 19 10:26 .
drwxr-xr-x. 85 nobody nobody      8192 Sep 19 10:37 ..
-rw-r--r--.  1 nobody nobody  10485760 Sep 19 10:25 00000000000000000000.index
-rw-r--r--.  1 nobody nobody 724167522 Sep 19 10:25 00000000000000000000.log
-rw-r--r--.  1 nobody nobody  10485756 Sep 19 10:25 00000000000000000000.timeindex
-rw-r--r--.  1 nobody nobody         0 Sep 19 10:15 leader-epoch-checkpoint
-rw-r--r--.  1 nobody nobody        43 Sep 19 10:15 partition.metadata

/demo/broker_demo_1/__a_demo_topic-2:
total 780016
drwxr-xr-x.  2 nobody nobody      4096 Sep 19 10:35 .
drwxr-xr-x. 85 nobody nobody      8192 Sep 19 10:37 ..
-rw-r--r--.  1 nobody nobody  10485760 Sep 19 10:36 00000000000000000000.index
-rw-r--r--.  1 nobody nobody 794575786 Sep 19 10:36 00000000000000000000.log
-rw-r--r--.  1 nobody nobody  10485756 Sep 19 10:36 00000000000000000000.timeindex
-rw-r--r--.  1 nobody nobody         0 Sep 19 10:35 leader-epoch-checkpoint
-rw-r--r--.  1 nobody nobody        43 Sep 19 10:35 partition.metadata
```

為何選擇適用於**Kafka**工作負載的**NetApp NFS**？

既然有解決方法可解決使用Kafka進行NFS儲存時的不實重命名問題、您就能建立健全的部署、讓NetApp ONTAP 的支援資源能夠滿足您的Kafka工作負載。這不僅能大幅降低營運成本、還能為您的Kafka叢集帶來下列效益：

- *降低Kafka代理商的CPU使用率。*使用分離式NetApp ONTAP 解決方案、將磁碟I/O作業與代理商分開、進而減少其CPU佔用空間。
- *更快的代理程式還原時間。*由於在ONTAP Kafka代理節點之間共享分離式NetApp支援、因此新的運算執行個體可在短時間內取代不良的代理程式、而無需重建資料。
- *儲存效率。*由於應用程式的儲存層現在是透過NetApp ONTAP 資源中心進行配置、因此客戶可以利用ONTAP 隨附於此功能的所有儲存效率優勢、例如即時資料壓縮、重複資料刪除和資料壓縮。

這些效益已在測試案例中經過測試和驗證、我們將在本節中詳細討論。

降低**Kafka**代理程式的**CPU**使用率

我們發現、當我們在兩個sperate Kafka叢集上執行類似工作負載時、整體CPU使用率比DAS低、這些叢集在技術規格上完全相同、但儲存技術卻不同。不僅是卡夫卡叢集使用ONTAP 率較低的整體CPU使用率、而且CPU使用率的增加、顯示出比DAS型卡夫卡叢集更為溫和的漸層。

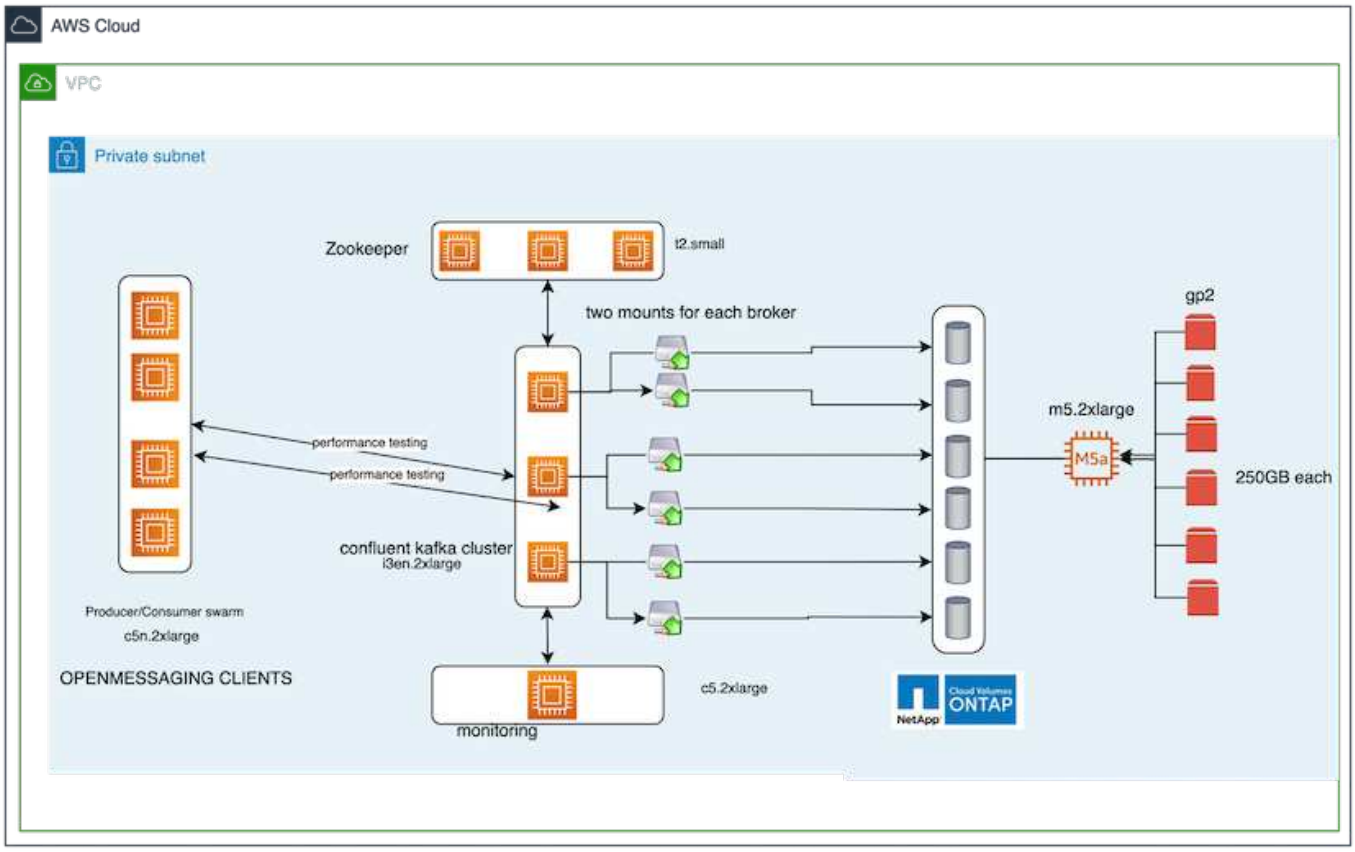
架構設定

下表顯示用於示範降低CPU使用率的環境組態。

平台元件	環境組態
Kafka 3.2.3基準測試工具：OpenMessaging	• 3個zookeepers–T2.small • 3個代理伺服器–i3en.2xLarge • 1 x Grafana–c5n.2xLarge • 4個製造商/消費者- c5n.2xLarge
所有節點上的作業系統	RHEL 8.7或更新版本
NetApp Cloud Volumes ONTAP 執行個體	單節點執行個體–M5.2xLarg

基準測試工具

本測試案例中使用的基準測試工具為 "OpenMessaging" 架構。OpenMessaging不受廠商限制、不受語言限制；它提供金融、電子商務、物聯網和巨量資料等產業準則、有助於跨異質系統和平台開發訊息和串流應用程式。下圖說明OpenMessaging用戶端與Kafka叢集之間的互動。



- *運算*我們使用三節點的Kafka叢集、其中三節點的zookeeper集合體執行於專屬的伺服器上。每個代理商都有兩個NFSv4.1掛載點、透過專屬的LIF連接至NetApp CVO執行個體上的單一Volume。
- *監控*我們使用兩個節點來搭配Prometheus-Grafana。為了產生工作負載、我們有一個獨立的三節點叢集、可產生和使用此Kafka叢集。
- * Storage。*我們使用單節點NetApp Cloud Volumes ONTAP 支援執行個體、並在執行個體上安裝六個250GB GP2 AWS-EBS磁碟區。然後、這些磁碟區會透過專用的LIF、以六個NFSv4.1磁碟區的形式呈現給Kafka叢集。

- *組態。*此測試案例中的兩個可設定元素為Kafka Brokers和OpenMessaging工作負載。
  - * Broker config.*為Kafka經紀人選擇了下列規格。我們在所有測量項目中都使用3個複寫係數、如下所示。

```
broker.id=1
advertised.listeners=PLAINTEXT://172.30.0.185:9092
log.dirs=/mnt/data-1
zookeeper.connect=172.30.0.13:2181,172.30.0.108:2181,172.30.0.253:2181
num.replica.fetchers=8
message.max.bytes=10485760
replica.fetch.max.bytes=10485760
num.network.threads=8
default.replication.factor=3
replica.lag.time.max.ms=100000000
replica.fetch.max.bytes=1048576
replica.fetch.wait.max.ms=500
num.replica.fetchers=1
replica.high.watermark.checkpoint.interval.ms=5000
fetch.purgatory.purge.interval.requests=1000
producer.purgatory.purge.interval.requests=1000
replica.socket.timeout.ms=30000
replica.socket.receive.buffer.bytes=65536
```

- * OpenMessaging基準測試（OMB）工作負載組態。*提供下列規格。我們指定目標生產商比率、重點如下。

```
name: 4 producer / 4 consumers on 1 topic
topics: 1
partitionsPerTopic: 100
messageSize: 1024
payloadFile: "payload/payload-1Kb.data"
subscriptionsPerTopic: 1
consumerPerSubscription: 4
producersPerTopic: 4
producerRate: 40000
consumerBacklogSizeGB: 0
testDurationMinutes: 5
```

測試方法

1. 建立了兩個類似的叢集、每個叢集都有自己的基準測試叢集。
   - 叢集**1.** NFS型Kafka叢集。
   - 叢集**2.** DAS型Kafka叢集。
2. 使用OpenMessaging命令、會在每個叢集上觸發類似的工作負載。

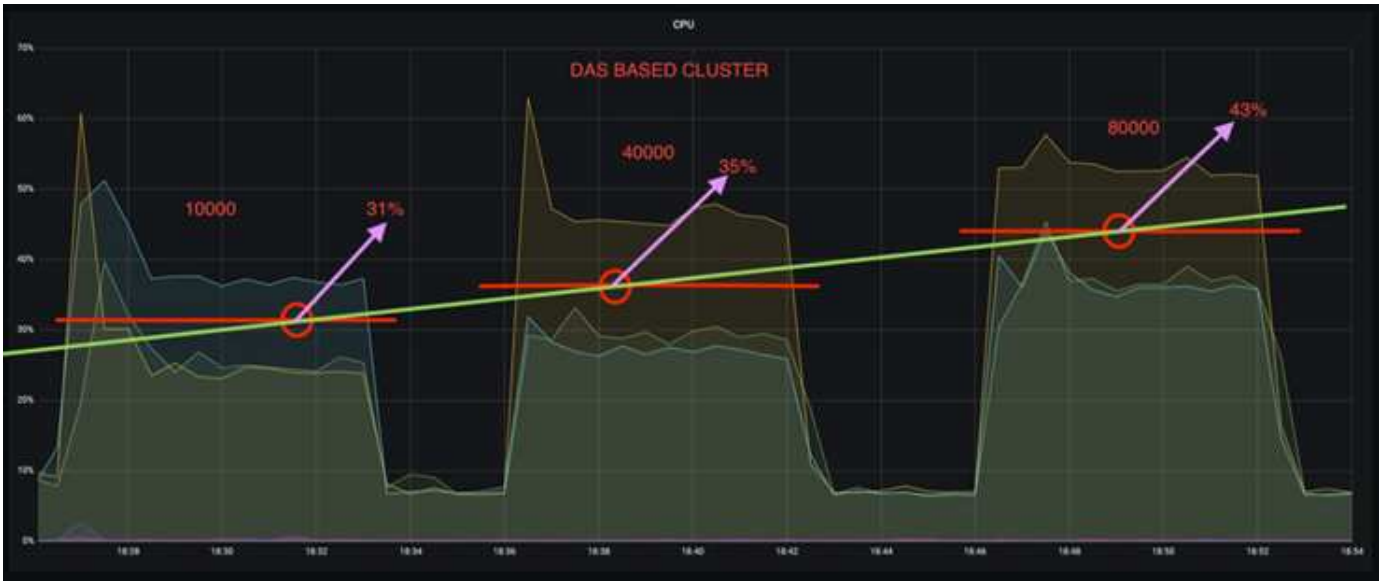
```
sudo bin/benchmark --drivers driver-kafka/kafka-group-all.yaml
workloads/1-topic-100-partitions-1kb.yaml
```

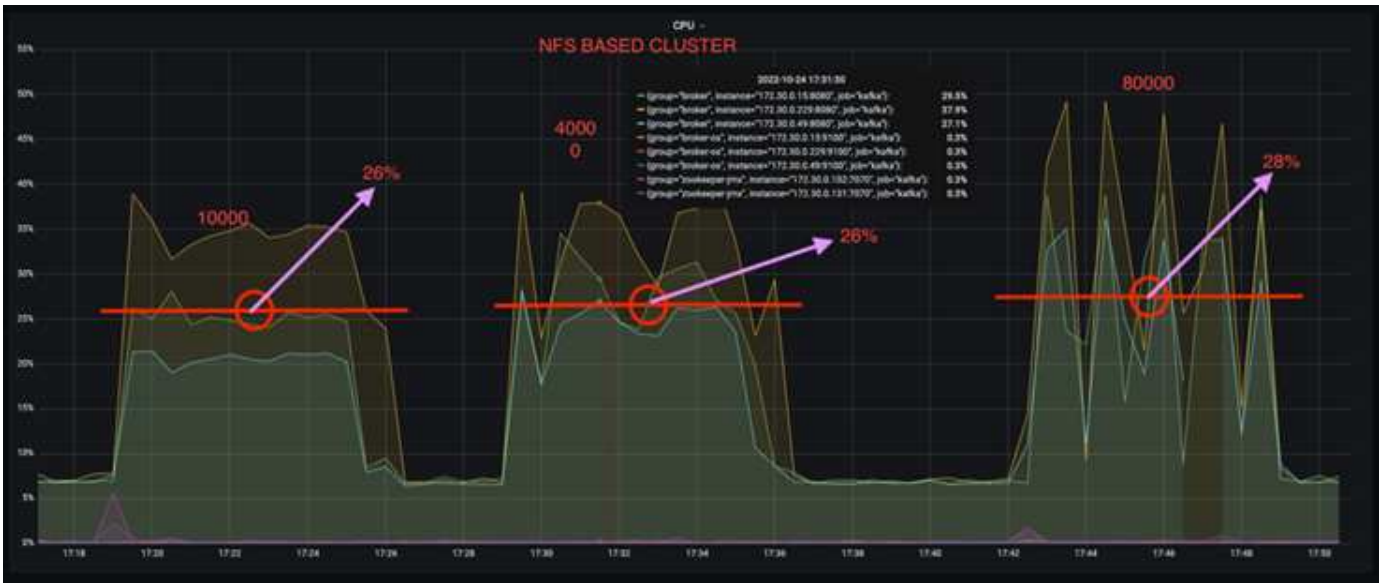
3. 產生率組態在四次迭代中增加、而CPU使用率則記錄在Grafana。產品出價設定為下列層級：
    - 10、000
    - 40、000
    - 80、000
    - 10、000

觀察

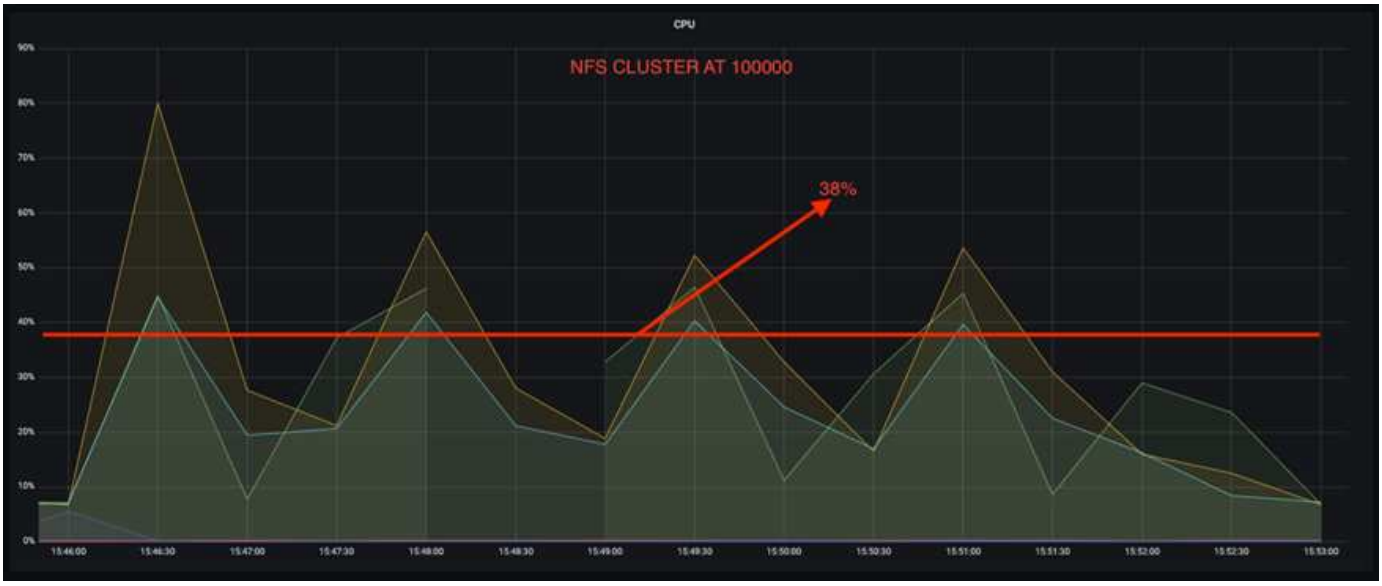
搭配Kafka使用NetApp NFS儲存設備有兩大主要效益：

- *您可以減少將近三分之一的CPU使用率。*相較於DAS SSD、NFS在類似工作負載下的整體CPU使用率較低；較低的產品使用率可節省5%、較高的產品使用率則可節省32%。
- 以較高的產生率、**CPU**使用率會減少三倍。*如預期、隨著生產率增加、CPU使用率會增加、因此CPU使用率會增加。不過、使用DAS的Kafka代理商的CPU使用率從較低的產品使用率的31 %增加到較高的產品使用率的70 %、增加39%。不過、有了NFS儲存後端、CPU使用率從26%增加到38%、增加12%。





此外、在100、000則訊息中、DAS顯示的CPU使用率比NFS叢集高。



更快速的代理程式還原

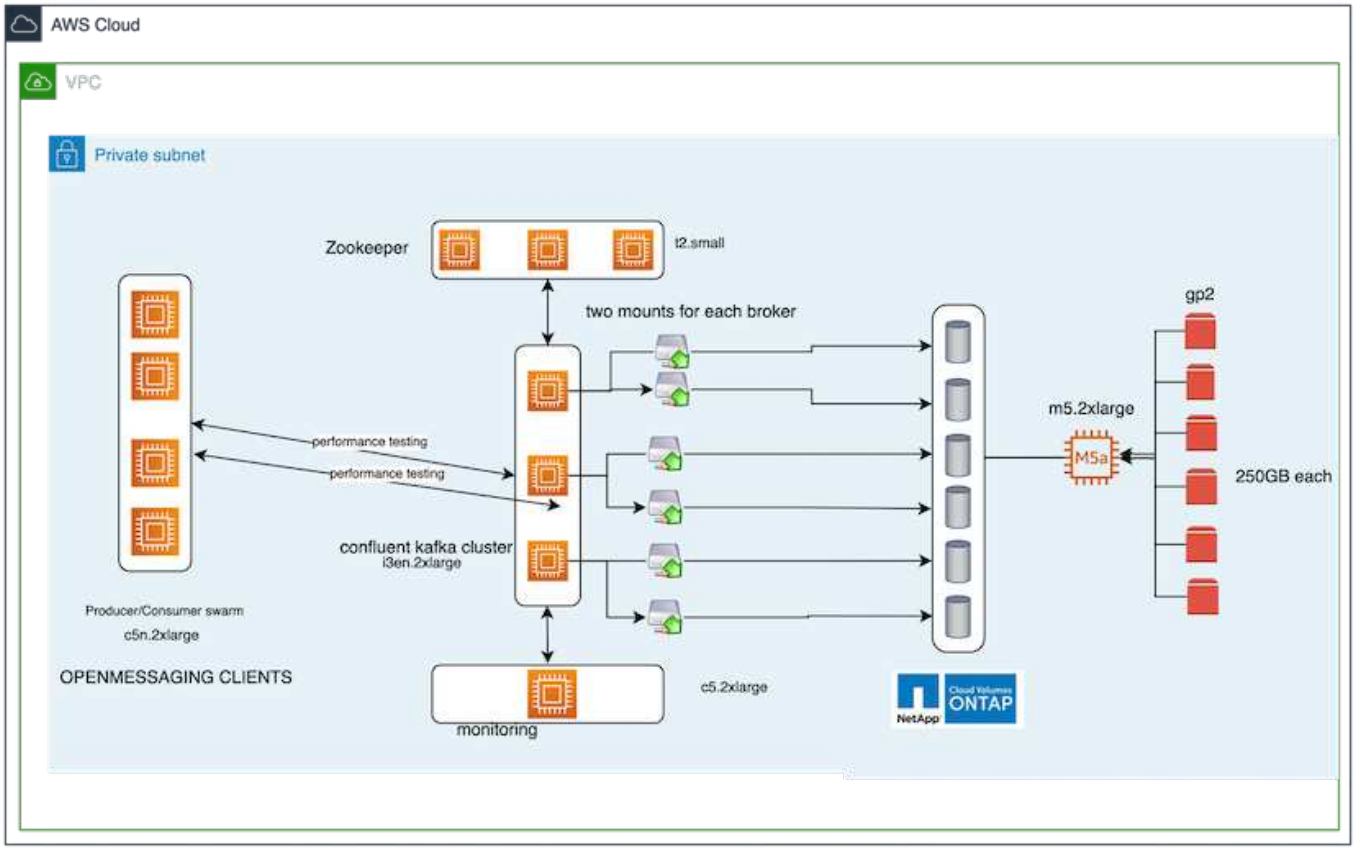
我們發現、Kafka代理商使用共享NetApp NFS儲存設備時、恢復速度更快。當Kafka叢集中的代理程式當機時、此代理程式可由具有相同代理程式ID的健全代理程式取代。在執行此測試案例時、我們發現在以DAS為基礎的Kafka叢集上、叢集會在新增的健全代理程式上重新建置資料、這相當耗時。在NetApp NFS型Kafka叢集的情況下、更換的代理程式會繼續從先前的記錄目錄讀取資料、並以更快的速度恢復。

架構設定

下表顯示使用NAS的Kafka叢集環境組態。

平台元件	環境組態
Kafka 3.2.3	• 3個zookeepers–T2.small • 3個代理伺服器–i3en.2xLarge • 1 x Grafana–c5n.2xLarge • 4個製造商/消費者- c5n.2xLarge • 1個備份Kafka節點–i3en.2xLarge
所有節點上的作業系統	RHEL8.7或更新版本
NetApp Cloud Volumes ONTAP 執行個體	單節點執行個體–M5.2xLarge

下圖說明NAS型Kafka叢集的架構。



- *運算。*三節點Kafka叢集、在專用伺服器上執行三節點zookeeper集合體。每個代理程式都有兩個NFS掛載點、可透過專屬LIF連接至NetApp CVO執行個體上的單一磁碟區。
- 監控 Prometheus-Grafana組合的兩個節點。為了產生工作負載、我們使用獨立的三節點叢集、可產生並使用此Kafka叢集。
- * Storage。*單節點NetApp Cloud Volumes ONTAP 效能實例、執行個體上安裝六個250GB GP2 AWS-EBS磁碟區。然後、這些磁碟區會透過專屬的LIF、以六個NFS磁碟區的形式呈現給Kafka叢集。
- * Broker組態。*此測試案例中的其中一個可設定元素是Kafka Broker。卡夫卡經紀公司選擇了下列規格。。`replica.lag.time.mx.ms` 設定為高值、因為這會決定從ISR清單中取出特定節點的速度。當您在不良和健全的節點之間切換時，您不希望將該代理ID排除在ISR清單之外。

```
broker.id=1
advertised.listeners=PLAINTEXT://172.30.0.185:9092
log.dirs=/mnt/data-1
zookeeper.connect=172.30.0.13:2181,172.30.0.108:2181,172.30.0.253:2181
num.replica.fetchers=8
message.max.bytes=10485760
replica.fetch.max.bytes=10485760
num.network.threads=8
default.replication.factor=3
replica.lag.time.max.ms=100000000
replica.fetch.max.bytes=1048576
replica.fetch.wait.max.ms=500
num.replica.fetchers=1
replica.high.watermark.checkpoint.interval.ms=5000
fetch.purgatory.purge.interval.requests=1000
producer.purgatory.purge.interval.requests=1000
replica.socket.timeout.ms=30000
replica.socket.receive.buffer.bytes=65536
```

測試方法

1. 建立了兩個類似的叢集：
   - 以EC2為基礎的匯合叢集。
   - NetApp NFS型的匯合叢集。
2. 建立一個待命的Kafka節點時、其組態與原始Kafka叢集的節點相同。
3. 在每個叢集上、都建立了範例主題、並在每個代理程式上填入約110 GB的資料。
   - *基於EC2的叢集。*會對應一個Kafka Broker資料目錄 `/mnt/data-2` （下圖為叢集1的Broler-1 [left終端機]）。
   - * NetApp NFS型叢集。*在NFS點上掛載Kafka Broker資料目錄 `/mnt/data` （下圖為叢集2的Broler-1 [右對講機]）。

4. 在每個叢集中、Brocher-1都會終止、以觸發失敗的Broker恢復程序。

5. 代理終止後、會將代理IP位址指派為次要IP給待命代理程式。這是必要的、因為Kafka叢集中的代理程式是由下列項目識別：
    - * IP位址。*指派方式是將故障的代理IP重新指派給待命代理程式。
    - * Broker ID。*這是在待命代理程式中設定的 `server.properties`。
6. 指派IP後、便會在待命代理程式上啟動Kafka服務。
7. 一段時間之後、伺服器記錄會被拉出、以檢查在叢集中的替換節點上建置資料所需的時間。

觀察

Kafka代理商的恢復速度快了將近九倍。相較於使用Kafka叢集中的DAS SSD、使用NetApp NFS共享儲存設備時、恢復故障代理節點所花的時間大幅加快。對於1TB的主題資料、DAS型叢集的恢復時間為48分鐘、而NetApp NFS型Kafka叢集的恢復時間則不到5分鐘。

我們觀察到、以EC2為基礎的叢集花了10分鐘在新的代理節點上重建110GB的資料、而以NFS為基礎的叢集則在3分鐘內完成恢復。我們也在「In（記錄）」中發現、EC2的分割區使用者偏移值為0、而在NFS叢集上、使用者偏移值則是從先前的代理程式中取得。

```
[2022-10-31 09:39:17,747] INFO [LogLoader partition=test-topic-51R3EWs-
0000-55, dir=/mnt/kafka-data/broker2] Reloading from producer snapshot and
rebuilding producer state from offset 583999 (kafka.log.UnifiedLog$)
[2022-10-31 08:55:55,170] INFO [LogLoader partition=test-topic-qbVsEZg-
0000-8, dir=/mnt/data-1] Loading producer state till offset 0 with message
format version 2 (kafka.log.UnifiedLog$)
```

DAS型叢集

1. 備份節點於08：55：53、730開始。

```
2  [2022-10-31 08:55:53,661] INFO Setting -D jdk.tls.rejectClientInitiatedRenegotia
3  [2022-10-31 08:55:53,727] INFO Registered signal handlers for TERM, INT, HUP (org
4  [2022-10-31 08:55:53,730] INFO starting (kafka.server.KafkaServer)
5  [2022-10-31 08:55:53,730] INFO Connecting to zookeeper on 172.30.0.17:2181,172.30
6  [2022-10-31 08:55:53,755] INFO [ZooKeeperClient Kafka server] Initializing a new
```

2. 資料重建程序於09：05：24、860結束。處理110 GB的資料大約需要10分鐘。

```
[2022-10-31 09:05:24,860] INFO [ReplicaFetcherManager on broker 1] Removed fetcher for
partitions HashSet(test-topic-qbVsEZg-0000-95, test-topic-qbVsEZg-0000-5,
test-topic-qbVsEZg-0000-41, test-topic-qbVsEZg-0000-23, test-topic-qbVsEZg-0000-11,
test-topic-qbVsEZg-0000-47, test-topic-qbVsEZg-0000-83, test-topic-qbVsEZg-0000-35,
test-topic-qbVsEZg-0000-89, test-topic-qbVsEZg-0000-71, test-topic-qbVsEZg-0000-53,
test-topic-qbVsEZg-0000-29, test-topic-qbVsEZg-0000-59, test-topic-qbVsEZg-0000-77,
test-topic-qbVsEZg-0000-65, test-topic-qbVsEZg-0000-17)
(kafka.server.ReplicaFetcherManager)
```

NFS型叢集

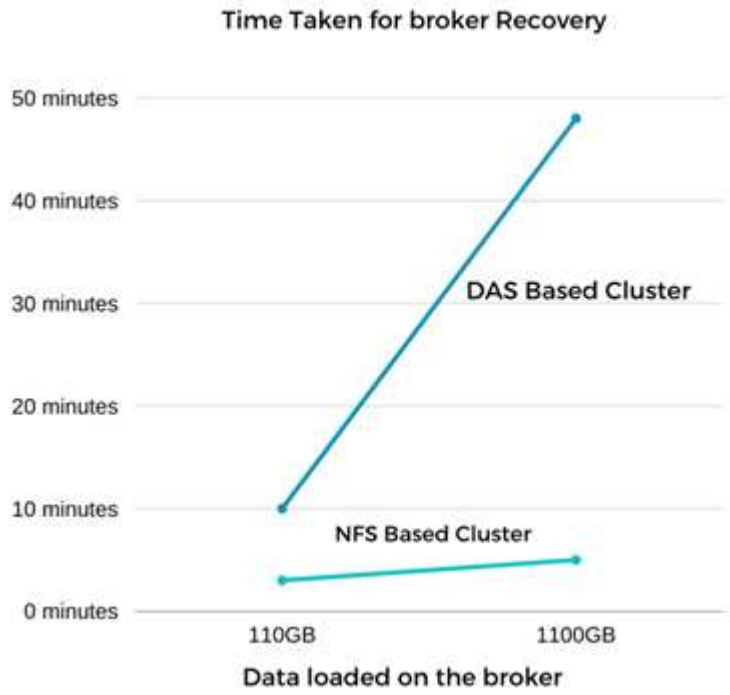
1. 備份節點於09：39：17、213開始。下方會強調顯示開始記錄項目。

```
[2022-10-31 09:39:17,142] INFO Setting -D jdk.tls.rejectClientInitiatedRenegotiati
[2022-10-31 09:39:17,211] INFO Registered signal handlers for TERM, INT, HUP (org.
[2022-10-31 09:39:17,213] INFO starting (kafka.server.KafkaServer)
[2022-10-31 09:39:17,214] INFO Connecting to zookeeper on 172.30.0.22:2181,172.30.
[2022-10-31 09:39:17,238] INFO [ZooKeeperClient Kafka server] Initializing a new s
[2022-10-31 09:39:17,244] INFO Client environment:zookeeper.version=3.6.3--6401e4a
[2022-10-31 09:39:17,244] INFO Client environment:host.name=ip-172-30-0-110.ec2.in
[2022-10-31 09:39:17,244] INFO Client environment:java.version=11.0.17 (org.apache
```

2. 資料重建程序於09：42：29、115結束。處理110 GB的資料大約需要3分鐘。

```
[2022-10-31 09:42:29,115] INFO [GroupMetadataManager brokerId=1] Finished loading offsets
and group metadata from __consumer_offsets-20 in 28478 milliseconds for epoch 3, of which
28478 milliseconds was spent in the scheduler.
(kafka.coordinator.group.GroupMetadataManager)
```

針對包含約1TB資料的代理商重複測試、DAS約需48分鐘、NFS約需3分鐘。結果如下圖所示。



儲存效率

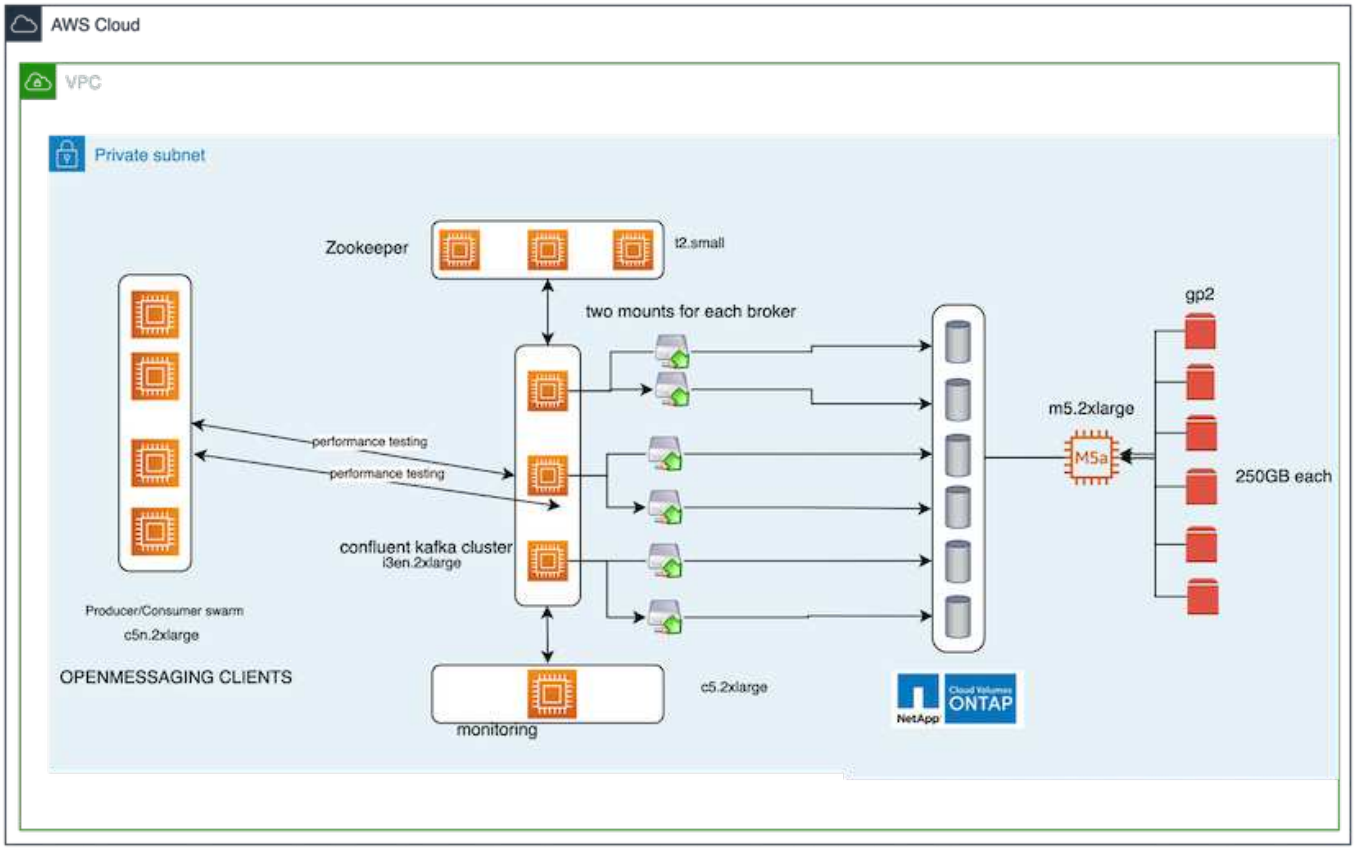
由於Kafka叢集的儲存層是透過NetApp ONTAP 供應、因此我們獲得ONTAP 了所有的NetApp儲存效率功能。測試結果是在安裝Cloud Volumes ONTAP 了NFS儲存設備的Kafka叢集上產生大量資料。我們可以看到ONTAP 、由於採用了一些功能、空間大幅縮減。

架構設定

下表顯示使用NAS的Kafka叢集環境組態。

平台元件	環境組態
Kafka 3.2.3	• 3個zookeepers–T2.small • 3個代理伺服器–i3en.2xLarge • 1 x Grafana–c5n.2xLarge • 4個製造商/消費者- c5n.2xLarge *
所有節點上的作業系統	RHEL8.7或更新版本
NetApp Cloud Volumes ONTAP 執行個體	單節點執行個體–M5.2xLarge

下圖說明NAS型Kafka叢集的架構。



- *運算*我們使用三節點的Kafka叢集、其中三節點的zookeeper集合體執行於專屬的伺服器上。每個代理商都有兩個NFS掛載點、可透過專屬LIF連接至NetApp CVO執行個體上的單一磁碟區。
- *監控*我們使用兩個節點來搭配Prometheus-Grafana。為了產生工作負載、我們使用了一個獨立的三節點叢集、可以產生和使用這個Kafka叢集。
- * Storage。*我們使用單節點NetApp Cloud Volumes ONTAP 支援執行個體、並在執行個體上安裝六個250GB GP2 AWS-EBS磁碟區。然後、這些磁碟區會透過專用的LIF、以六個NFS磁碟區的形式呈現給Kafka叢集。
- *組態。*此測試案例中的可設定元素是Kafka仲介。

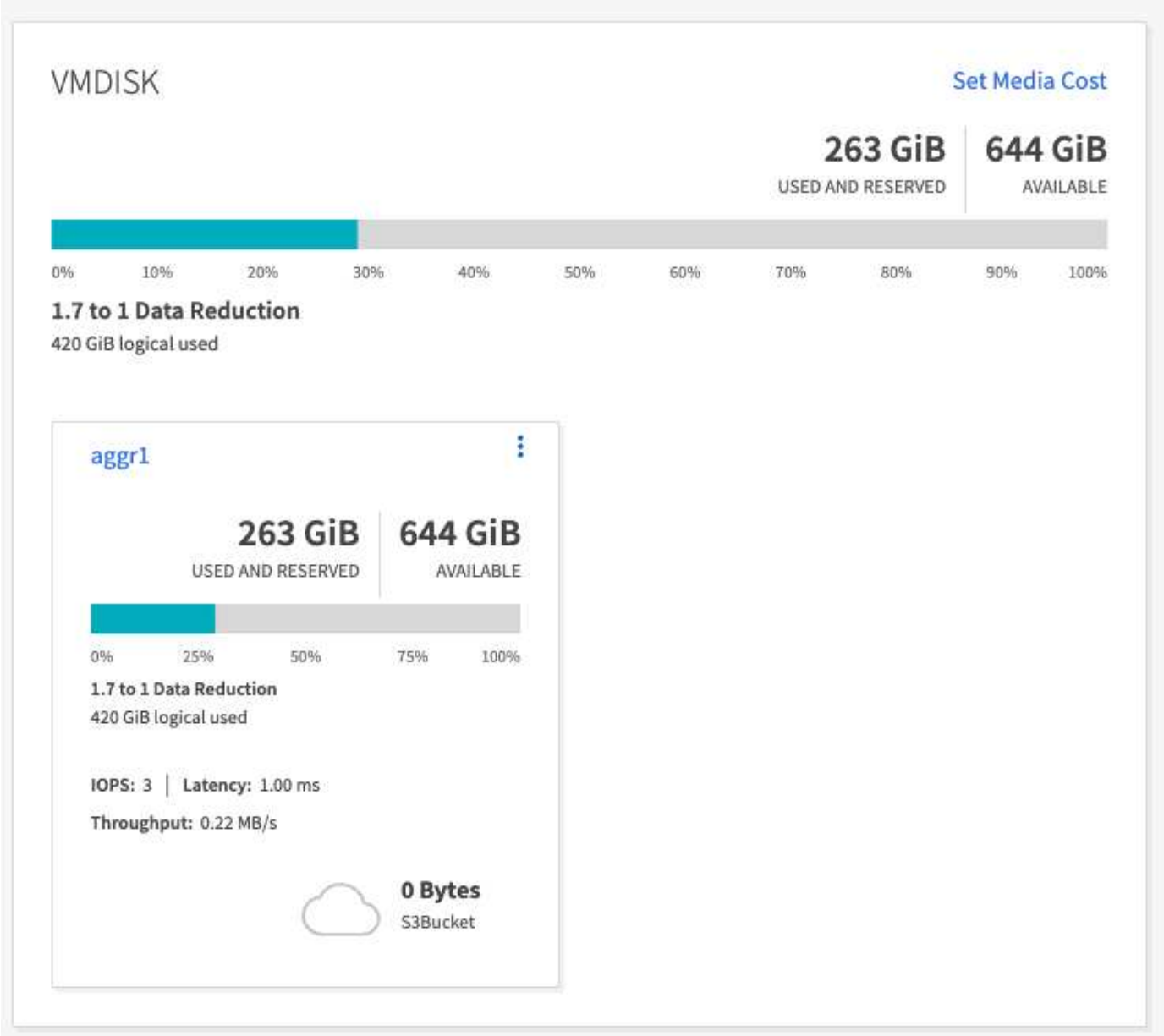
在生產商端點關閉壓縮功能、讓生產商產生高處理量。儲存效率則由運算層來處理。

測試方法

1. 卡夫卡叢集已配置上述規格。
2. 在叢集上、使用OpenMessaging基準測試工具產生約350 GB的資料。
3. 工作負載完成後、會使用ONTAP NetApp System Manager和CLI收集儲存效率統計資料。

觀察

對於使用OMB工具產生的資料、我們發現空間節約約33%、儲存效率比為1.70:1。如下圖所示、所產生資料所使用的邏輯空間為420.3GB、用於保存資料的實體空間為281.7GB。



```
shantanuCVOinstancenew::> df -h -S

Warning: The "-S" parameter is deprecated and may be removed in a future release. To show the efficiency ratio use "aggr show-efficiency"
         command.
Filesystem               used      total-saved    %total-saved    deduplicated    %deduplicated    compressed    %compressed  Vserver
/vol/vol0/             7319MB               0B              0%              0B               0%            0B             0%  shantanuCVOinstancenew-01
/vol/kafka_vol/         281GB            138GB             33%           138GB              33%            0B             0%  svm_shantanuCVOinstancenew
/vol/svm_shantanuCVOinstancenew_root/
                        660KB               0B              0%              0B               0%            0B             0%  svm_shantanuCVOinstancenew
3 entries were displayed.
```

```
                        Name of the Aggregate: aggr1
                 Node where Aggregate Resides: shantanuCVOinstancenew-01
                Total Storage Efficiency Ratio: 1.70:1
Total Data Reduction Efficiency Ratio Without Snapshots: 1.70:1
Total Data Reduction Efficiency Ratio without snapshots and flexclones: 1.70:1
           Logical Space Used for All Volumes: 420.3GB
          Physical Space Used for All Volumes: 281.7GB
```

AWS的效能總覽與驗證

安裝在NetApp NFS上的儲存層Kafka叢集、是以AWS雲端效能為基準。基準測試範例將於下列各節中說明。

採用NetApp Cloud Volumes ONTAP 技術的AWS雲端中的Kafka（高可用度配對與單一節點）

採用NetApp Cloud Volumes ONTAP 功能的Kafka叢集（HA配對）是AWS雲端效能的基準測試。以下各節將說明此基準測試。

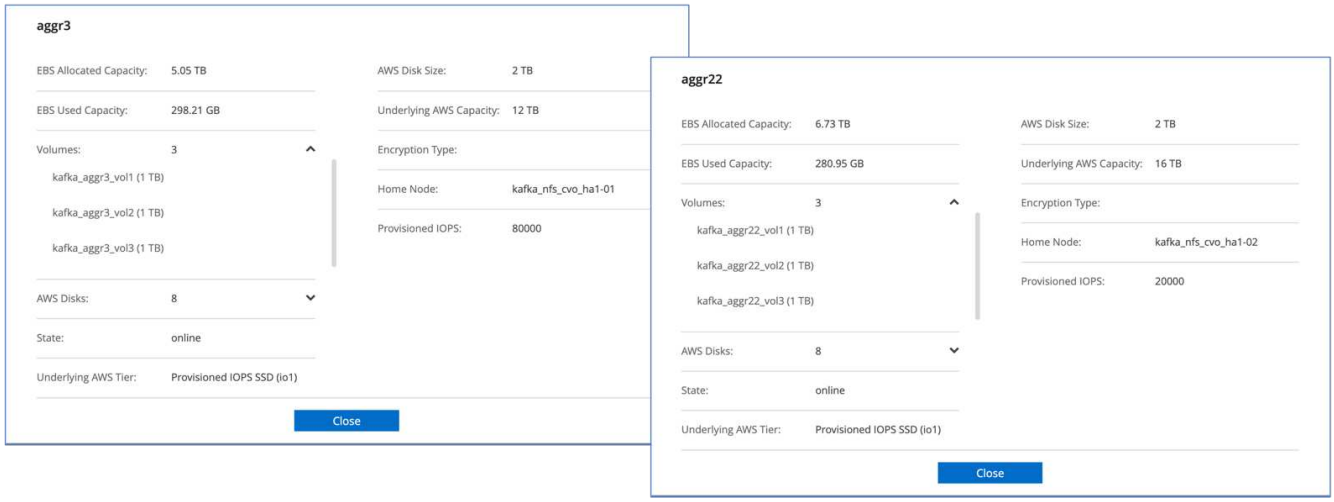
架構設定

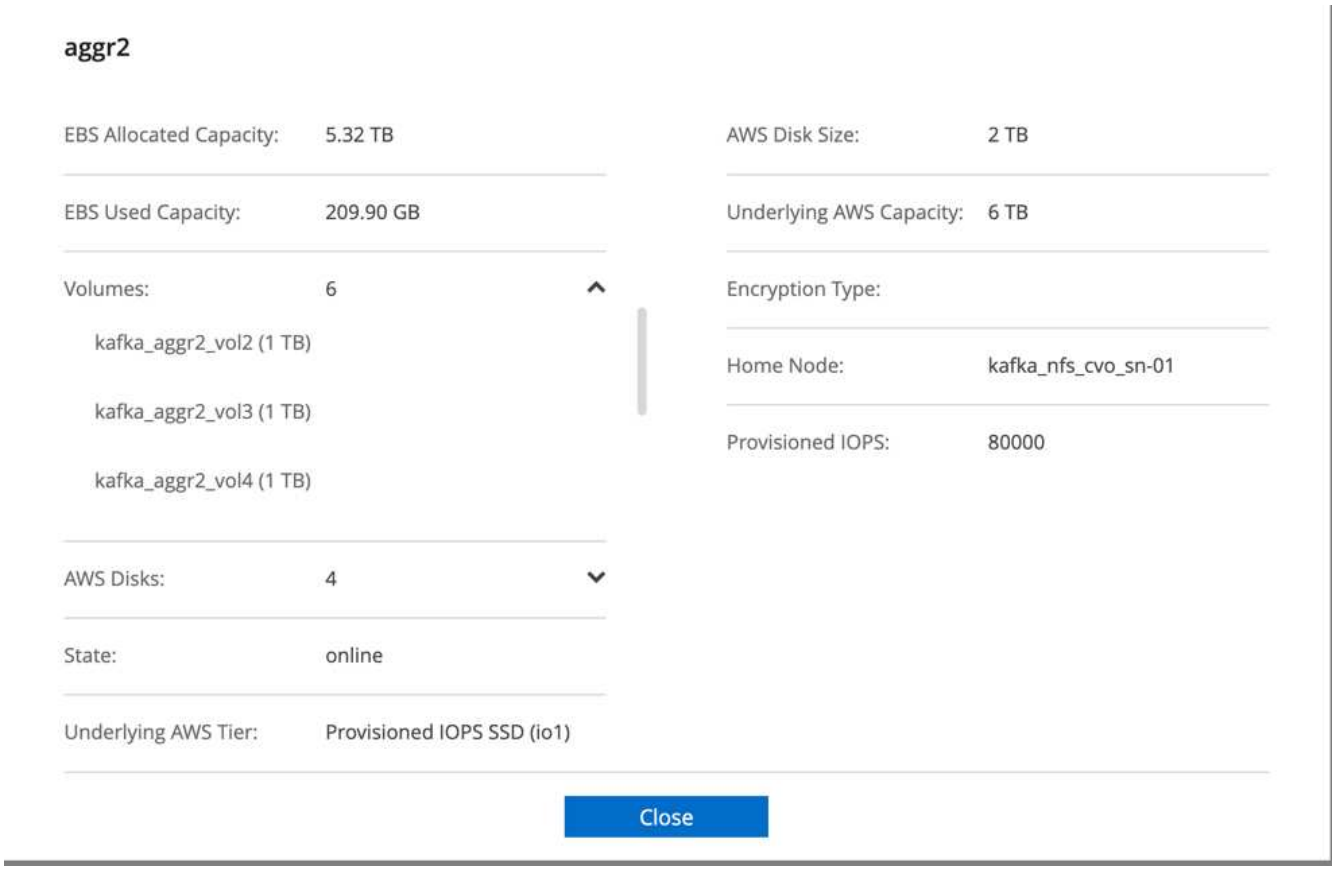
下表顯示使用NAS的Kafka叢集環境組態。

平台元件	環境組態
Kafka 3.2.3	• 3個zookeepers–T2.small • 3個代理伺服器–i3en.2xLarge • 1 x Grafana–c5n.2xLarge • 4個製造商/消費者- c5n.2xLarge *
所有節點上的作業系統	RHEL8.6.
NetApp Cloud Volumes ONTAP 執行個體	HA配對執行個體–m5dn.12xLargex 2節點單一節點執行個體- m5dn.12xLargex 1節點

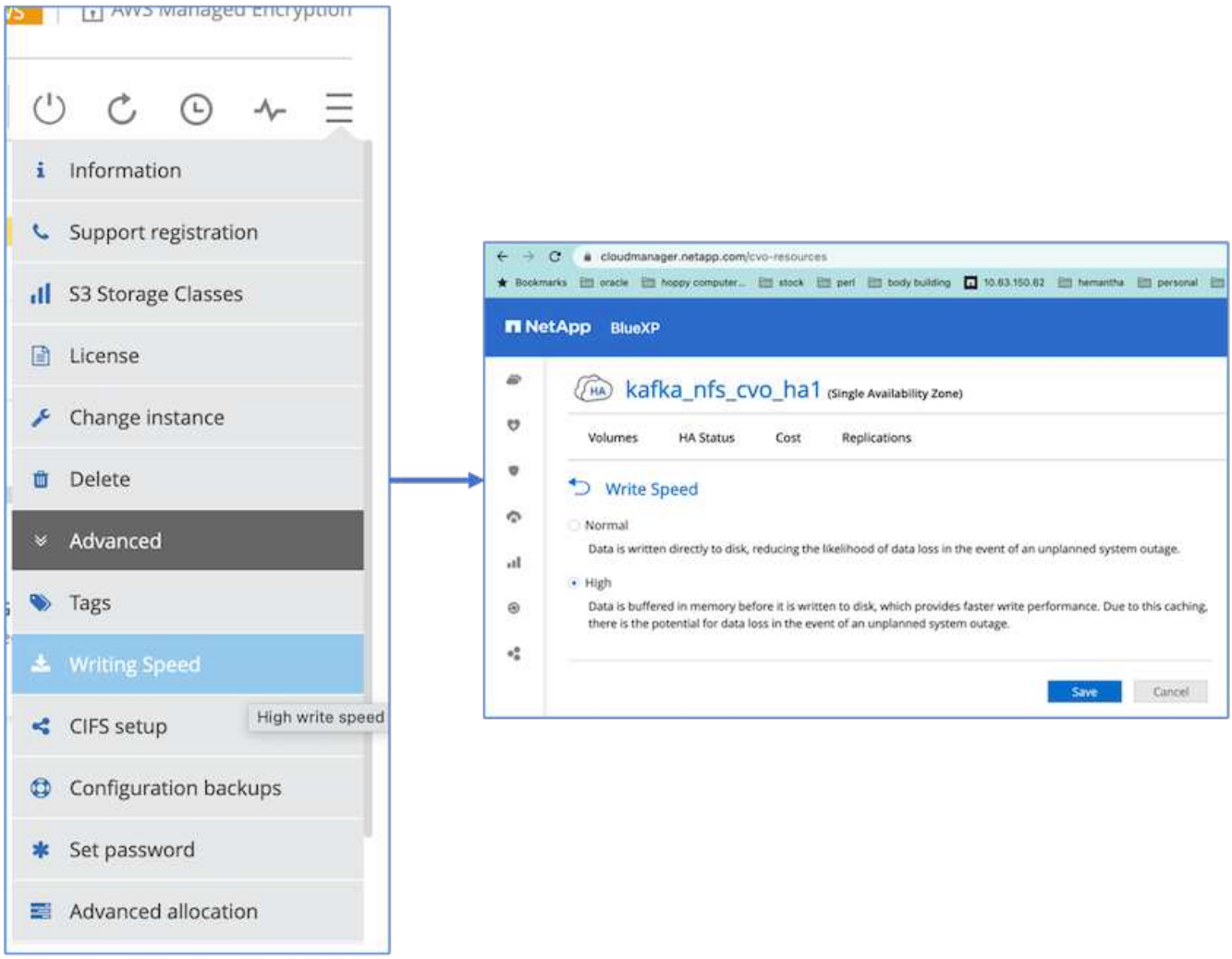
NetApp叢集Volume ONTAP 不全

1. 針對「樣片HA配對」、我們在每個儲存控制器的每個集合體上建立了兩個集合體、並有三個磁碟區Cloud Volumes ONTAP 。對於單Cloud Volumes ONTAP 一的實體節點、我們會在一個集合體中建立六個磁碟區。



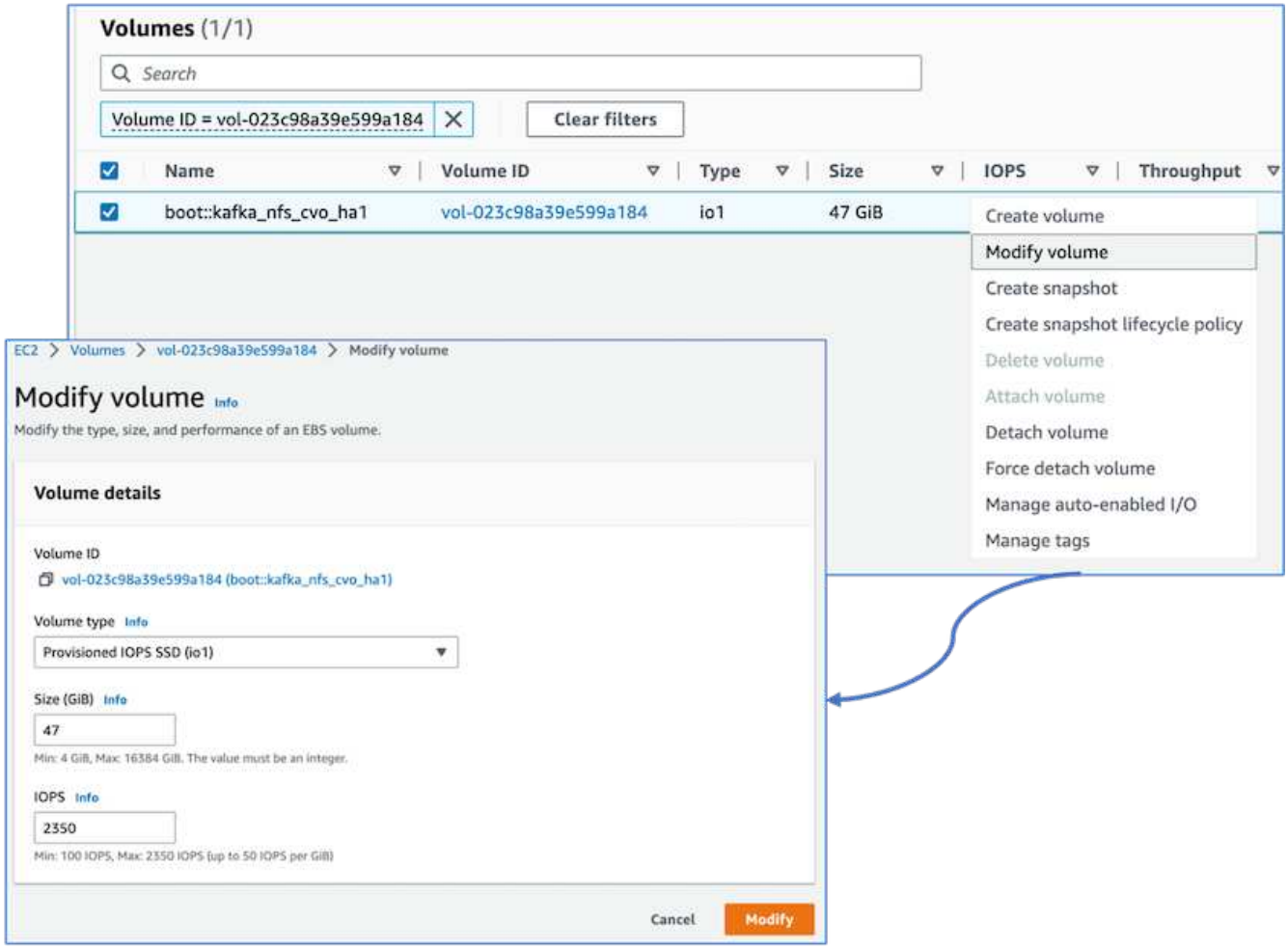


2. 為了提升網路效能、我們同時為HA配對和單一節點啟用高速網路功能。



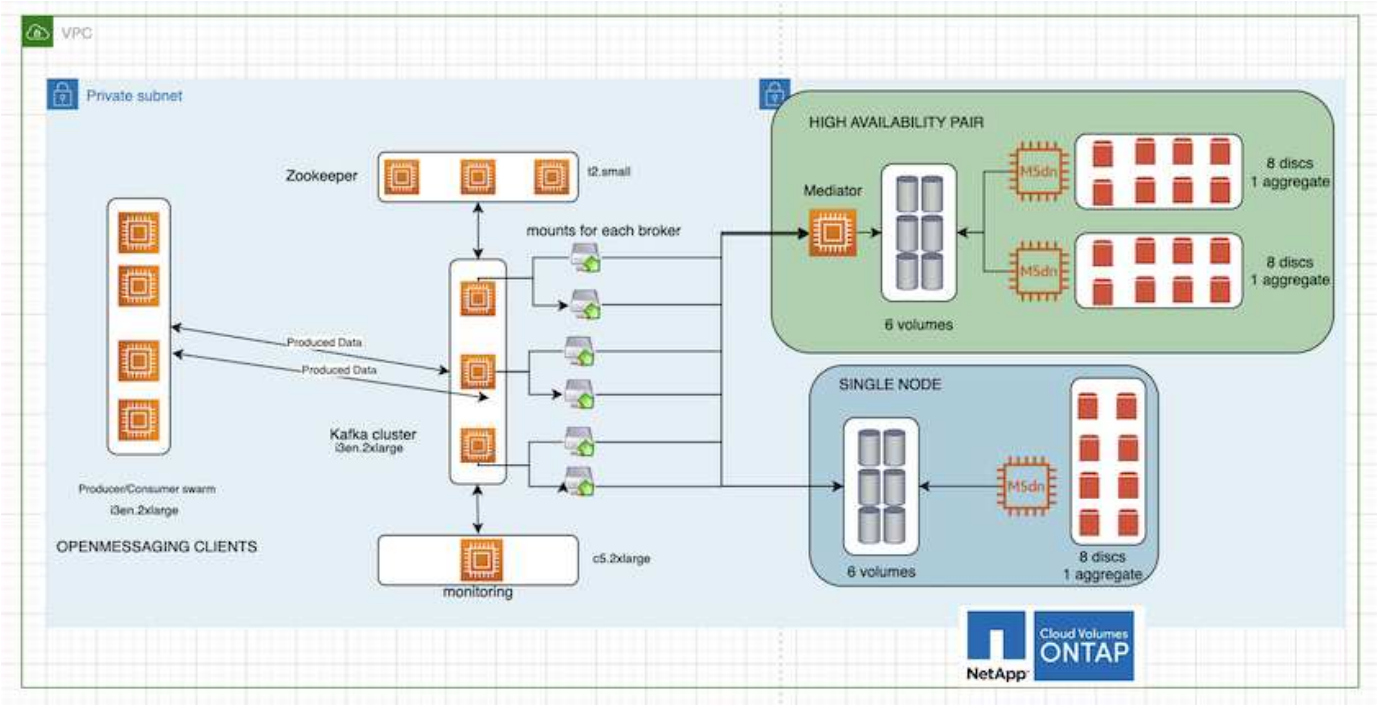
3. 我們注意到ONTAP 、由於不支援IOPS、Cloud Volumes ONTAP 所以我們將IOPS變更為2350、以供支援整個核心磁碟區使用。以47GB大小為基礎的根Volume磁碟Cloud Volumes ONTAP 。下列ONTAP 支援HA配對的支援功能使用下列支援功能、單一節點也適用相同的步驟。

```
statistics start -object vnvram -instance vnvram -counter
backing_store_iops -sample-id sample_555
kafka_nfs_cvo_ha1::*> statistics show -sample-id sample_555
Object: vnvram
Instance: vnvram
Start-time: 1/18/2023 18:03:11
End-time: 1/18/2023 18:03:13
Elapsed-time: 2s
Scope: kafka_nfs_cvo_ha1-01
    Counter                                                     Value
    -------------------------------- --------------------------------
    backing_store_iops                                           1479
Object: vnvram
Instance: vnvram
Start-time: 1/18/2023 18:03:11
End-time: 1/18/2023 18:03:13
Elapsed-time: 2s
Scope: kafka_nfs_cvo_ha1-02
    Counter                                                     Value
    -------------------------------- --------------------------------
    backing_store_iops                                           1210
2 entries were displayed.
kafka_nfs_cvo_ha1::*>
```



下圖說明NAS型Kafka叢集的架構。

- *運算*我們使用三節點的Kafka叢集、其中三節點的zookeeper集合體執行於專屬的伺服器上。每個代理商都有兩個NFS掛載點、Cloud Volumes ONTAP 透過專屬的LIF、連接到位於整個過程的單一Volume。
- *監控*我們使用兩個節點來搭配Prometheus-Grafana。為了產生工作負載、我們使用了一個獨立的三節點叢集、可以產生和使用這個Kafka叢集。
- * Storage。* Cloud Volumes ONTAP 我們使用HA配對的不二執行個體、在執行個體上掛載一個6TB GP3 AWS-EBS磁碟區。然後使用NFS掛載將磁碟區匯出至Kafka代理程式。



OpenMessage基準測試組態

1. 為了提升NFS效能、我們需要在NFS伺服器和NFS用戶端之間建立更多的網路連線、而NFS用戶端可以使用nconnect建立。執行下列命令、以nconnect選項在代理節點上掛載NFS磁碟區：

```
[root@ip-172-30-0-121 ~]# cat /etc/fstab
UUID=eaa1f38e-de0f-4ed5-a5b5-2fa9db43bb38/xfsdefaults00
/dev/nvme1n1 /mnt/data-1 xfs defaults,noatime,nodiscard 0 0
/dev/nvme2n1 /mnt/data-2 xfs defaults,noatime,nodiscard 0 0
172.30.0.233:/kafka_aggr3_vol1 /kafka_aggr3_vol1 nfs
defaults,nconnect=16 0 0
172.30.0.233:/kafka_aggr3_vol2 /kafka_aggr3_vol2 nfs
defaults,nconnect=16 0 0
172.30.0.233:/kafka_aggr3_vol3 /kafka_aggr3_vol3 nfs
defaults,nconnect=16 0 0
172.30.0.242:/kafka_aggr22_vol1 /kafka_aggr22_vol1 nfs
defaults,nconnect=16 0 0
172.30.0.242:/kafka_aggr22_vol2 /kafka_aggr22_vol2 nfs
defaults,nconnect=16 0 0
172.30.0.242:/kafka_aggr22_vol3 /kafka_aggr22_vol3 nfs
defaults,nconnect=16 0 0
[root@ip-172-30-0-121 ~]# mount -a
[root@ip-172-30-0-121 ~]# df -h
Filesystem                       Size  Used Avail Use% Mounted on
devtmpfs                          31G     0   31G   0% /dev
tmpfs                             31G  249M   31G   1% /run
tmpfs                             31G     0   31G   0% /sys/fs/cgroup
/dev/nvme0n1p2                    10G  2.8G  7.2G  28% /
/dev/nvme1n1                     2.3T  248G  2.1T  11% /mnt/data-1
/dev/nvme2n1                     2.3T  245G  2.1T  11% /mnt/data-2
172.30.0.233:/kafka_aggr3_vol1   1.0T   12G 1013G   2% /kafka_aggr3_vol1
172.30.0.233:/kafka_aggr3_vol2   1.0T  5.5G 1019G   1% /kafka_aggr3_vol2
172.30.0.233:/kafka_aggr3_vol3   1.0T  8.9G 1016G   1% /kafka_aggr3_vol3
172.30.0.242:/kafka_aggr22_vol1  1.0T  7.3G 1017G   1%
/kafka_aggr22_vol1
172.30.0.242:/kafka_aggr22_vol2  1.0T  6.9G 1018G   1%
/kafka_aggr22_vol2
172.30.0.242:/kafka_aggr22_vol3  1.0T  5.9G 1019G   1%
/kafka_aggr22_vol3
tmpfs                            6.2G     0  6.2G   0% /run/user/1000
[root@ip-172-30-0-121 ~]#
```

2. 檢查Cloud Volumes ONTAP 內部的網路連線。下列ONTAP 支援從單Cloud Volumes ONTAP 一支援節點使用下列支援功能。同樣的步驟也適用於Cloud Volumes ONTAP 此功能。

```
Last login time: 1/20/2023 00:16:29
kafka_nfs_cvo_sn::> network connections active show -service nfs*
-fields remote-host
node                cid        vserver              remote-host
------------------- ---------- -------------------- ------------
```

```
kafka_nfs_cvo_sn-01 2315762628 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762629 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762630 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762631 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762632 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762633 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762634 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762635 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762636 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762637 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762639 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762640 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762641 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762642 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762643 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762644 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762645 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762646 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762647 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762648 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762649 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762650 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762651 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762652 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762653 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762656 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762657 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762658 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762659 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762660 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762661 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762662 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762663 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762664 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762665 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762666 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762667 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762668 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762669 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762670 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762671 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762672 svm_kafka_nfs_cvo_sn 172.30.0.72
kafka_nfs_cvo_sn-01 2315762673 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762674 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762676 svm_kafka_nfs_cvo_sn 172.30.0.121
kafka_nfs_cvo_sn-01 2315762677 svm_kafka_nfs_cvo_sn 172.30.0.223
```

```
kafka_nfs_cvo_sn-01 2315762678 svm_kafka_nfs_cvo_sn 172.30.0.223
kafka_nfs_cvo_sn-01 2315762679 svm_kafka_nfs_cvo_sn 172.30.0.223
48 entries were displayed.

kafka_nfs_cvo_sn::>
```

3. 我們使用下列Kafka `server.properties` 所有的Kafka經紀人都能提供Cloud Volumes ONTAP。`log.dirs` 每個代理的屬性各不相同、其餘屬性則適用於代理程式。若為Broker1、則為 `log.dirs` 價值如下：

```
[root@ip-172-30-0-121 ~]# cat /opt/kafka/config/server.properties
broker.id=0
advertised.listeners=PLAINTEXT://172.30.0.121:9092
#log.dirs=/mnt/data-1/d1,/mnt/data-1/d2,/mnt/data-1/d3,/mnt/data-
2/d1,/mnt/data-2/d2,/mnt/data-2/d3
log.dirs=/kafka_aggr3_vol1/broker1,/kafka_aggr3_vol2/broker1,/kafka_aggr
3_vol3/broker1,/kafka_aggr22_vol1/broker1,/kafka_aggr22_vol2/broker1,/ka
fka_aggr22_vol3/broker1
zookeeper.connect=172.30.0.12:2181,172.30.0.30:2181,172.30.0.178:2181
num.network.threads=64
num.io.threads=64
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600
num.partitions=1
num.recovery.threads.per.data.dir=1
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
replica.fetch.max.bytes=524288000
background.threads=20
num.replica.alter.log.dirs.threads=40
num.replica.fetchers=20
[root@ip-172-30-0-121 ~]#
```

   - 若為Broker2 `log.dirs` 屬性值如下：

```
log.dirs=/kafka_aggr3_vol1/broker2,/kafka_aggr3_vol2/broker2,/kafka_a
ggr3_vol3/broker2,/kafka_aggr22_vol1/broker2,/kafka_aggr22_vol2/broke
r2,/kafka_aggr22_vol3/broker2
```

   - 若為Broker3 `log.dirs` 屬性值如下：

```
log.dirs=/kafka_aggr3_vol1/broker3,/kafka_aggr3_vol2/broker3,/kafka_a
ggr3_vol3/broker3,/kafka_aggr22_vol1/broker3,/kafka_aggr22_vol2/broke
r3,/kafka_aggr22_vol3/broker3
```

4. 對於單Cloud Volumes ONTAP 一的支援節點、卡夫卡（Kafka） `servers.properties` 與Cloud Volumes ONTAP 不包括在內的其他不相同 `log.dirs` 屬性。
   - 若為Broker1、則為 `log.dirs` 價值如下：

```
log.dirs=/kafka_aggr2_vol1/broker1,/kafka_aggr2_vol2/broker1,/kafka_a
ggr2_vol3/broker1,/kafka_aggr2_vol4/broker1,/kafka_aggr2_vol5/broker1
,/kafka_aggr2_vol6/broker1
```

   - 若為Broker2 `log.dirs` 價值如下：

```
log.dirs=/kafka_aggr2_vol1/broker2,/kafka_aggr2_vol2/broker2,/kafka_a
ggr2_vol3/broker2,/kafka_aggr2_vol4/broker2,/kafka_aggr2_vol5/broker2
,/kafka_aggr2_vol6/broker2
```

   - 若為Broker3 `log.dirs` 屬性值如下：

```
log.dirs=/kafka_aggr2_vol1/broker3,/kafka_aggr2_vol2/broker3,/kafka_a
ggr2_vol3/broker3,/kafka_aggr2_vol4/broker3,/kafka_aggr2_vol5/broker3
,/kafka_aggr2_vol6/broker3
```

5. OMB中的工作負載會設定下列內容： `(/opt/benchmark/workloads/1-topic-100-partitions-1kb.yaml)`。

```
topics: 4
partitionsPerTopic: 100
messageSize: 32768
useRandomizedPayloads: true
randomBytesRatio: 0.5
randomizedPayloadPoolSize: 100
subscriptionsPerTopic: 1
consumerPerSubscription: 80
producersPerTopic: 40
producerRate: 1000000
consumerBacklogSizeGB: 0
testDurationMinutes: 5
```

- `messageSize` 可能因使用案例而異。在效能測試中、我們使用3K。

我們使用OMB的兩個不同驅動程式：同步或處理量、來產生Kafka叢集上的工作負載。

- 用於Sync驅動程式內容的yaml檔案如下 `(/opt/benchmark/driver- kafka/kafka-sync.yaml)`：

```
name: Kafka
driverClass:
io.openmessaging.benchmark.driver.kafka.KafkaBenchmarkDriver
# Kafka client-specific configuration
replicationFactor: 3
topicConfig: |
  min.insync.replicas=2
  flush.messages=1
  flush.ms=0
commonConfig: |

bootstrap.servers=172.30.0.121:9092,172.30.0.72:9092,172.30.0.223:909
2
producerConfig: |
  acks=all
  linger.ms=1
  batch.size=1048576
consumerConfig: |
  auto.offset.reset=earliest
  enable.auto.commit=false
  max.partition.fetch.bytes=10485760
```

- 用於處理量驅動程式內容的yaml檔案如下 `(/opt/benchmark/driver- kafka/kafka-throughput.yaml)`：

```
name: Kafka
driverClass:
io.openmessaging.benchmark.driver.kafka.KafkaBenchmarkDriver
# Kafka client-specific configuration
replicationFactor: 3
topicConfig: |
  min.insync.replicas=2
commonConfig: |

bootstrap.servers=172.30.0.121:9092,172.30.0.72:9092,172.30.0.223:909
2
  default.api.timeout.ms=1200000
  request.timeout.ms=1200000
producerConfig: |
  acks=all
  linger.ms=1
  batch.size=1048576
consumerConfig: |
  auto.offset.reset=earliest
  enable.auto.commit=false
  max.partition.fetch.bytes=10485760
```

測試方法

1. 根據上述規格、我們使用Terraform和Ansible來配置Kafka叢集。Terraform是用來為Kafka叢集使用AWS執行個體來建置基礎架構、Ansible則是在這些執行個體上建置Kafka叢集。
2. 使用上述工作負載組態和Sync驅動程式觸發OMB工作負載。

```
Sudo bin/benchmark -drivers driver-kafka/kafka- sync.yaml workloads/1-
topic-100-partitions-1kb.yaml
```

3. 另一個工作負載是透過具有相同工作負載組態的處理量驅動程式觸發。

```
sudo bin/benchmark -drivers driver-kafka/kafka-throughput.yaml
workloads/1-topic-100-partitions-1kb.yaml
```

觀察

使用兩種不同類型的驅動程式來產生工作負載、以基準測試在NFS上執行的Kafka執行個體效能。驅動程式之間的差異在於記錄排清內容。

若為Cloud Volumes ONTAP 「解決方案」配對：

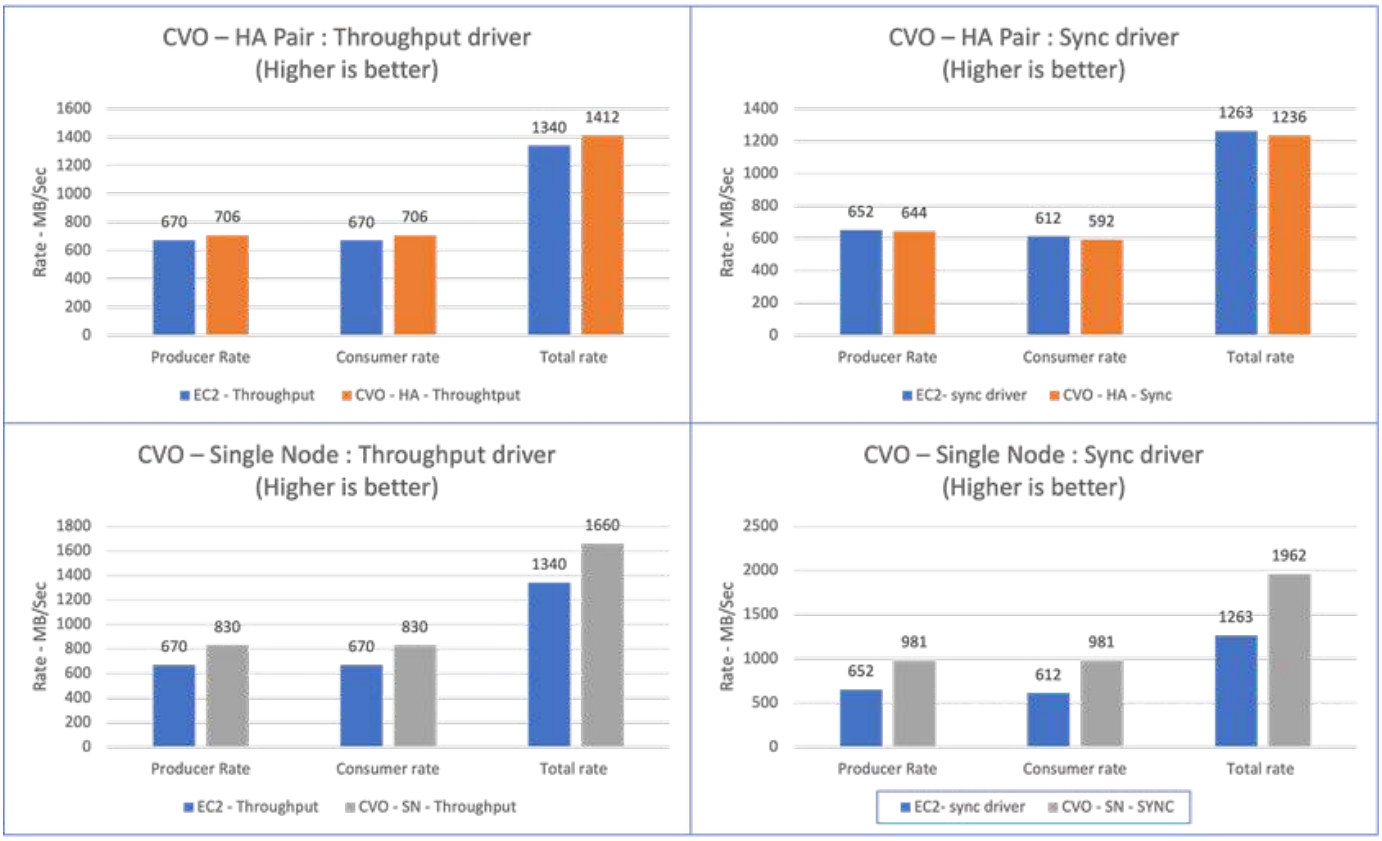
- Sync驅動程式持續產生的總處理量：約1236 Mbps。
- 為處理量驅動程式產生的總處理量：尖峰約1412 Mbps。

對於單Cloud Volumes ONTAP 一的節點：

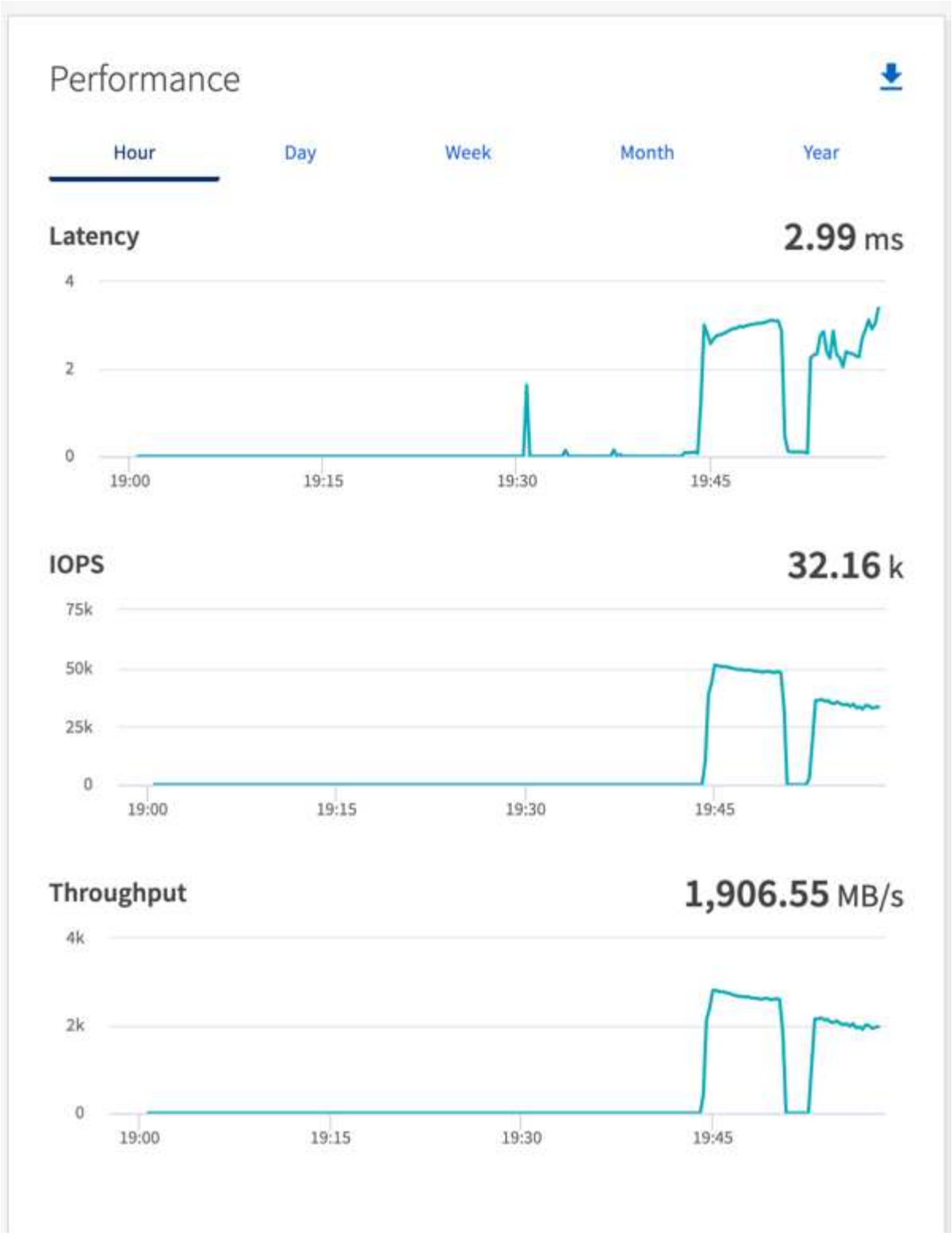
- Sync驅動程式持續產生的總處理量：約1962MBps。
- 處理量驅動程式產生的總處理量：尖峰約1660MBps

同步處理驅動程式可在記錄立即排入磁碟時產生一致的處理量、而處理量驅動程式則會在大量將記錄提交至磁碟時產生大量處理量。

這些處理量編號是針對指定的AWS組態所產生。為了達到更高的效能需求、可以進一步擴充和調整執行個體類型、以獲得更好的處理量。總處理量或總處理率是生產者和使用者速率的組合。



執行處理量或同步驅動程式基準測試時、請務必檢查儲存處理量。



AWS FSX for NetApp ONTAP 的效能概觀與驗證

NetApp NFS 上安裝儲存層的 Kafka 叢集、是 AWS FSX for NetApp ONTAP 效能的基準測試。基準測試範例將於下列各節中說明。

AWS FSX for NetApp ONTAP 中的 Apache Kafka

網路檔案系統（ NFS ）是廣泛使用的網路檔案系統、用於儲存大量資料。在大多數組織中、像 Apache Kafka 這樣的串流應用程式越來越多地產生資料。這些工作負載需要擴充性、低延遲、以及具備現代化儲存功能的健全資料擷取架構。為了實現即時分析並提供可據以行動的洞見、需要設計完善且效能優異的基礎架構。

Kafka by 的設計可搭配 POSIX 相容的檔案系統使用、並仰賴檔案系統來處理檔案作業、但在 NFSv3 檔案系統上儲存資料時、 Kafka Broker NFS 用戶端可以與本地檔案系統（例如 XFS 或 ext4 ）不同地解譯檔案作業。常見的例子是 NFS 愚蠢重新命名、導致 Kafka Brokers 在擴充叢集和重新分配分割區時失敗。為了因應這項挑戰、 NetApp 已更新開放原始碼 Linux NFS 用戶端、現在已在 RHEL8.7 、 RHEL9.1 中普遍提供變更、並從目前的適用於 NetApp ONTAP 版本 ONTAP 9.12.1 的 FSX 中獲得支援。

Amazon FSX for NetApp ONTAP 可在雲端中提供完全託管、可擴充且高效能的 NFS 檔案系統。適用於 NetApp ONTAP 的 FSX 上的 Kafka 資料可擴充以處理大量資料、並確保容錯能力。NFS 可為重要且敏感的資料集提供集中化的儲存管理與資料保護。

這些增強功能可讓 AWS 客戶在 AWS 運算服務上執行 Kafka 工作負載時、善用適用於 NetApp ONTAP 的 FSX 。這些效益包括：
* 降低 CPU 使用率、以縮短 I/O 等待時間
* 更快的 Kafka 代理程式恢復時間。
* 可靠性與效率。
* 擴充性與效能。
* 多可用性區域可用度。
* 資料保護。

AWS FSX for NetApp ONTAP 的效能概觀與驗證

安裝在NetApp NFS上的儲存層Kafka叢集、是以AWS雲端效能為基準。基準測試範例將於下列各節中說明。

AWS FSX for NetApp ONTAP 中的 Kafka

採用 AWS FSX for NetApp ONTAP 的 Kafka 叢集、在 AWS 雲端中以效能為基準。以下各節將說明此基準測試。

架構設定

下表顯示使用 AWS FSX for NetApp ONTAP 的 Kafka 叢集環境組態。

平台元件	環境組態
Kafka 3.2.3	• 3個zookeepers–T2.small • 3個代理伺服器–i3en.2xLarge • 1 x Grafana–c5n.2xLarge • 4個製造商/消費者- c5n.2xLarge *
所有節點上的作業系統	RHEL8.6.
AWS FSX for NetApp ONTAP	具有 4GB/ 秒處理量和 160000 IOPS 的多 AZ

NetApp FSX for NetApp ONTAP 設定

1. 在我們的初始測試中、我們為 NetApp ONTAP 檔案系統建立了 FSX 、容量為 2TB 、傳輸量為每秒 2GB 的 40000 IOPs 。

```
[root@ip-172-31-33-69 ~]# aws fsx create-file-system --region us-east-2
--storage-capacity 2048 --subnet-ids <desired subnet 1> subnet-<desired
subnet 2> --file-system-type ONTAP --ontap-configuration
DeploymentType=MULTI_AZ_HA_1,ThroughputCapacity=2048,PreferredSubnetId=<
desired primary subnet>,FsxAdminPassword=<new
password>,DiskIopsConfiguration="{Mode=USER_PROVISIONED,Iops=40000"}
```

   在我們的範例中、我們透過 AWS CLI 部署適用於 NetApp ONTAP 的 FSX 。您必須視需要在環境中進一步自訂命令。此外、也可透過 AWS 主控台部署和管理適用於 NetApp ONTAP 的 FSX 、以減少命令列輸入、提供更輕鬆、更精簡的部署體驗。

   FSX for NetApp ONTAP 的文件資料、我們測試區域（美國東部 -1 ）每秒 2 GB 處理量檔案系統的最高 IOPS 為 8 、 000 IOPS 。NetApp ONTAP 檔案系統的 FSX 總 IOPS 為 16 、 000 IOPS 、需要每秒 4 GB 的處理量部署、我們將在本文件稍後說明。

   如需適用於 NetApp ONTAP 效能規格的 FSX 詳細資訊、請隨時造訪 AWS FSX for NetApp ONTAP 文件： https://docs.aws.amazon.com/fsx/latest/ONTAPGuide/performance.html 。

   您可以在這裡找到適用於 FSX 「 cree-file-system 」的詳細命令列語法： https://docs.aws.amazon.com/cli/latest/reference/fsx/create-file-system.html

   例如、您可以指定特定的 KMS 金鑰、而非指定 KMS 金鑰時所使用的預設 AWS FSX 主要金鑰。

2. 為 NetApp ONTAP 檔案系統建立 FSX 時、請等到 JSON 將檔案系統描述如下之後、「生命週期」狀態變更為「可用」：

```
[root@ip-172-31-33-69 ~]# aws fsx describe-file-systems  --region us-
east-1 --file-system-ids fs-02ff04bab5ce01c7c
```

3. 與 fsxadmin 使用者登入適用於 NetApp ONTAP SSH 的 FSX 、以驗證認證：
   Fsxadmin 是建立時適用於 NetApp ONTAP 檔案系統之 FSX 的預設管理帳戶。fsxadmin 的密碼是我們在步驟 1 中完成的第一次在 AWS 主控台或 AWS CLI 中建立檔案系統時所設定的密碼。

```
[root@ip-172-31-33-69 ~]# ssh fsxadmin@198.19.250.244
The authenticity of host '198.19.250.244 (198.19.250.244)' can't be
established.
ED25519 key fingerprint is
SHA256:mgCyRXJfWRc2d/jOjFbMBsUcYOWjxoIky0ltHvVDL/Y.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '198.19.250.244' (ED25519) to the list of
known hosts.
(fsxadmin@198.19.250.244) Password:

This is your first recorded login.
```

4. 認證通過驗證後、請在適用於 NetApp ONTAP 檔案系統的 FSX 上建立儲存虛擬機器

```
[root@ip-172-31-33-69 ~]# aws fsx --region us-east-1 create-storage-
virtual-machine --name svmkafkatest --file-system-id fs-
02ff04bab5ce01c7c
```

儲存虛擬機器（ SVM ）是一部隔離式檔案伺服器、具有自己的管理認證和端點、可用於管理和存取適用於 NetApp ONTAP 磁碟區的 FSX 中的資料、並提供適用於 NetApp ONTAP 多租戶的 FSX 。

5. 設定主要儲存虛擬機器之後、請將 SSH 移至新建立的 NetApp ONTAP 檔案系統 FSX 、並使用以下範例命令在儲存虛擬機器中建立 Volume 、同樣地、我們也會建立 6 個磁碟區來進行此驗證。根據我們的驗證結果、保留預設成分（ 8 ）或以下成分、以提供更好的 Kafka 效能。

```
FsxId02ff04bab5ce01c7c::*> volume create -volume kafkafsxN1 -state
online -policy default -unix-permissions ---rwxr-xr-x -junction-active
true -type RW -snapshot-policy none  -junction-path /kafkafsxN1 -aggr
-list aggr1
```

6. 我們的磁碟區需要額外的容量來進行測試。將磁碟區的大小延伸至 2TB 、並安裝在連接路徑上。

```
FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN1 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN1" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN2 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN2" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN3 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN3" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN4 -new-size +2TB
```

```
vol size: Volume "svmkafkatest:kafkafsxN4" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN5 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN5" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume size -volume kafkafsxN6 -new-size +2TB
vol size: Volume "svmkafkatest:kafkafsxN6" size set to 2.10t.

FsxId02ff04bab5ce01c7c::*> volume show -vserver svmkafkatest -volume *
Vserver   Volume       Aggregate    State      Type       Size
Available Used%
--------- ------------ ------------ ---------- ---- ----------
---------- -----
svmkafkatest
          kafkafsxN1   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN2   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN3   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN4   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN5   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          kafkafsxN6   -            online     RW       2.10TB
1.99TB    0%
svmkafkatest
          svmkafkatest_root
                       aggr1        online     RW          1GB
968.1MB    0%
7 entries were displayed.

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN1 -junction
-path /kafkafsxN1

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN2 -junction
-path /kafkafsxN2

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN3 -junction
-path /kafkafsxN3
```

```
FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN4 -junction
-path /kafkafsxN4

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN5 -junction
-path /kafkafsxN5

FsxId02ff04bab5ce01c7c::*> volume mount -volume kafkafsxN6 -junction
-path /kafkafsxN6
```

在適用於 NetApp ONTAP 的 FSX 中、磁碟區可以精簡配置。在我們的範例中、擴充磁碟區總容量超過檔案系統總容量、因此我們需要擴充檔案系統總容量、才能解除鎖定額外的資源配置磁碟區容量、我們將在下一步中示範。

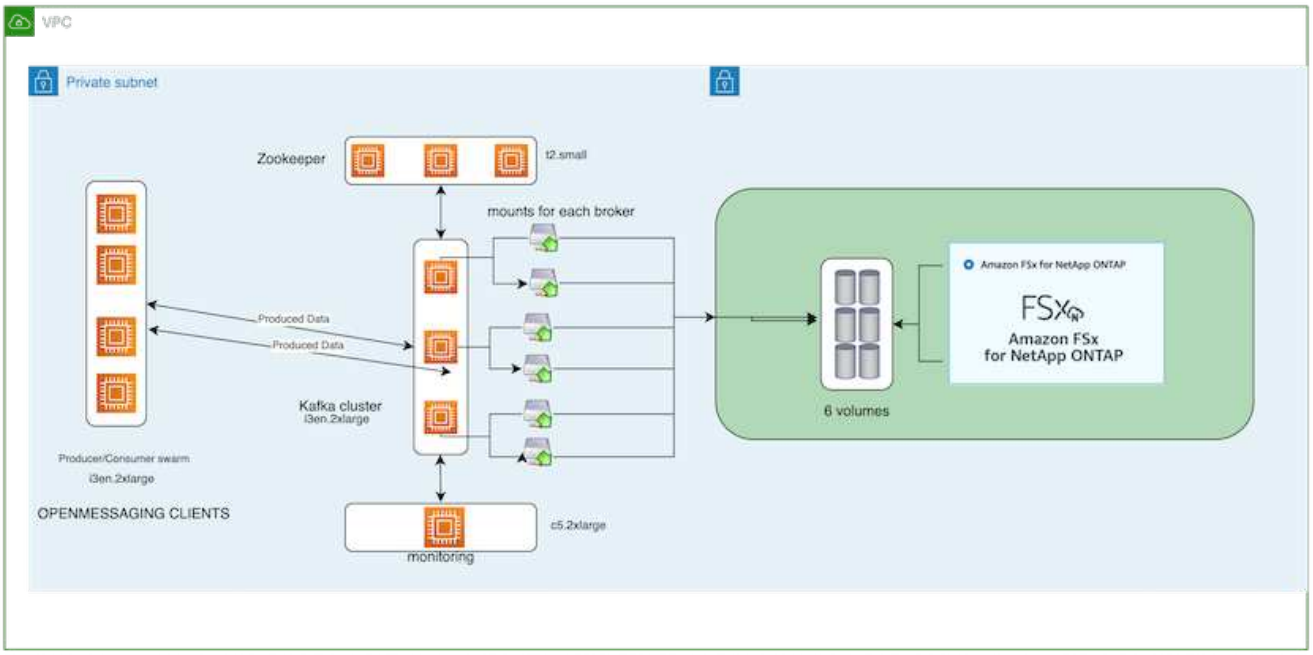
7. 接下來、為了提升效能與容量、我們將 NetApp ONTAP 處理量容量的 FSX 從每秒 2 GB 擴充至每秒 4 GB 、 IOPS 擴充至 160000 、容量則擴充至 5 TB

```
[root@ip-172-31-33-69 ~]# aws fsx update-file-system --region us-east-1
--storage-capacity 5120 --ontap-configuration
'ThroughputCapacity=4096,DiskIopsConfiguration={Mode=USER_PROVISIONED,Io
ps=160000}' --file-system-id fs-02ff04bab5ce01c7c
```

您可以在這裡找到適用於 FSX 「update-file-system 」的詳細命令列語法：
https://docs.aws.amazon.com/cli/latest/reference/fsx/update-file-system.html

8. NetApp ONTAP 磁碟區的 FSX 會在 Kafka Brokers 中使用 nconnect 和預設選項進行掛載

下圖顯示我們的 FSX for NetApp ONTAP 卡夫卡叢集的最終架構：



- 運算：我們使用三節點的 Kafka 叢集、在專用伺服器上執行三節點的 zookeeper 群集。每個代理程式都

有六個 NFS 掛載點、可連接至 FSX for NetApp ONTAP 執行個體上的六個磁碟區。

- 監控。我們使用兩個節點作為 Prometheus-Grafana 組合。為了產生工作負載、我們使用了一個獨立的三節點叢集、可以產生和使用這個Kafka叢集。
- 儲存設備。我們使用適用於 NetApp ONTAP 的 FSX 搭配六個 2TB 磁碟區。然後使用 NFS 掛載將該 Volume 匯出至 Kafka 代理程式。 NetApp ONTAP 磁碟區的 FSX 會在 Kafka 代理程式中以 16 個 nconnect 工作階段和預設選項掛載。

OpenMessage 基準測試組態。

我們使用的組態與 NetApp Cloud Volumes ONTAP 相同、其詳細資料如下：
https://docs.netapp.com/us-en/netapp-solutions/data-analytics/kafka-nfs-performance-overview-and-validation-in-aws.html#architectural-setup

測試方法

1. Kafka 叢集是根據上述規格、使用 terraform 和 Ansible 來進行佈建。Terraform 用於使用適用於 Kafka 叢集的 AWS 執行個體來建置基礎架構、 Ansible 則在其上建置 Kafka 叢集。
2. 使用上述工作負載組態和Sync驅動程式觸發OMB工作負載。

```
sudo bin/benchmark -drivers driver-kafka/kafka-sync.yaml workloads/1-topic-100-partitions-1kb.yaml
```

3. 另一個工作負載是透過具有相同工作負載組態的處理量驅動程式觸發。

```
sudo bin/benchmark -drivers driver-kafka/kafka-throughput.yaml workloads/1-topic-100-partitions-1kb.yaml
```

觀察

使用兩種不同類型的驅動程式來產生工作負載、以基準測試在NFS上執行的Kafka執行個體效能。驅動程式之間的差異在於記錄排清內容。

對於 Kafka Replication factor 1 和適用於 NetApp ONTAP 的 FSX ：

- 同步驅動程式一致產生的總處理量：約 3218 Mbps 、尖峰效能約 3652 Mbps 。
- 輸送量驅動程式一致產生的總處理量：約 3679 Mbps 、尖峰效能約 3908 Mbps 。

適用於具有複寫係數 3 的 Kafka 和適用於 NetApp ONTAP 的 FSX ：

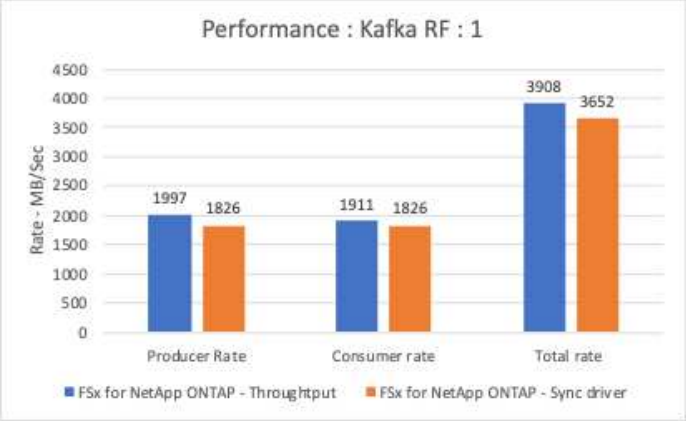
- 同步驅動程式一致產生的總處理量：約 1252 Mbps 、尖峰效能約 1382 Mbps 。
- 輸送量驅動程式一致產生的總處理量：約 1218 Mbps 、尖峰效能約 1328 Mbps 。

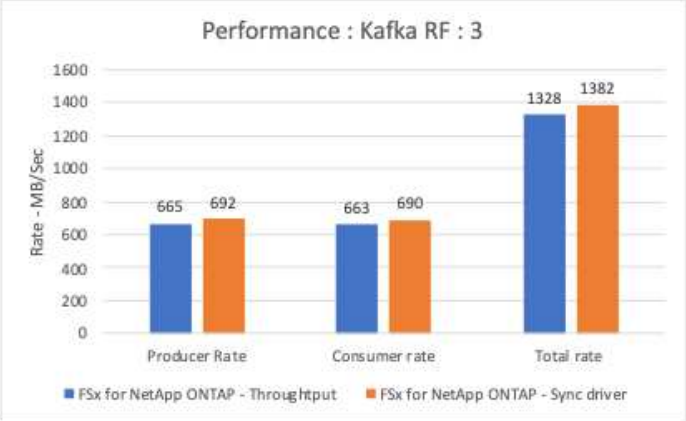
在 Kafka 複寫係數 3 中、 NetApp ONTAP 的 FSX 執行讀寫作業三次、在 Kafka 複寫係數 1 中、 NetApp ONTAP 的讀寫作業是在 FSX 上執行一次、因此在兩次驗證中、 我們能夠達到每秒 4GB 的最大處理量。

同步處理驅動程式可在記錄立即排入磁碟時產生一致的處理量、而處理量驅動程式則會在大量將記錄提交至磁碟

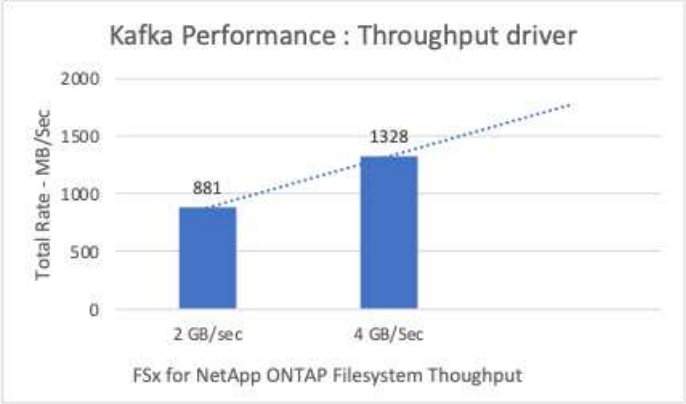
時產生大量處理量。

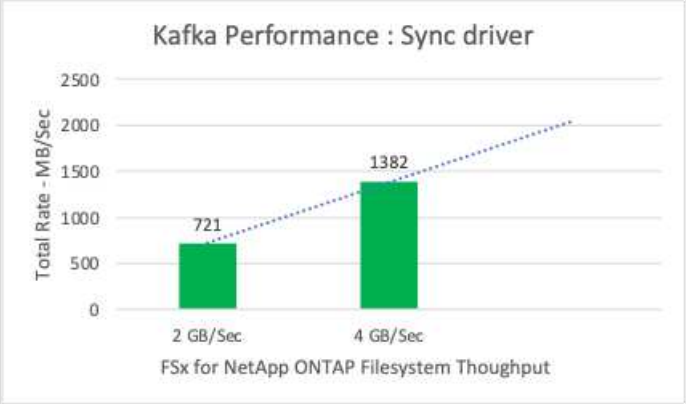
這些處理量編號是針對指定的AWS組態所產生。為了達到更高的效能需求、可以進一步擴充和調整執行個體類型、以獲得更好的處理量。總處理量或總處理率是生產者和使用者速率的組合。



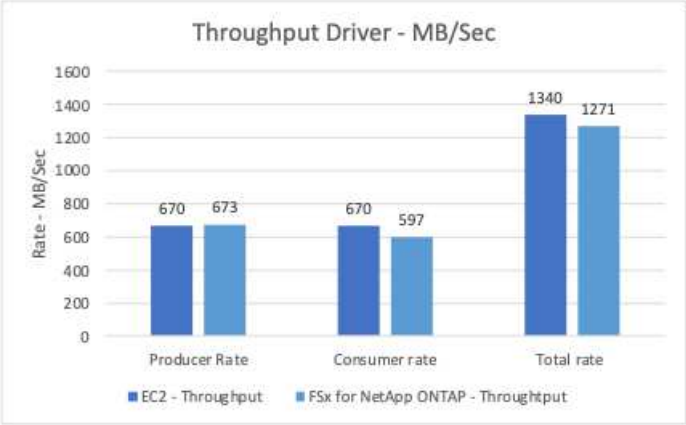


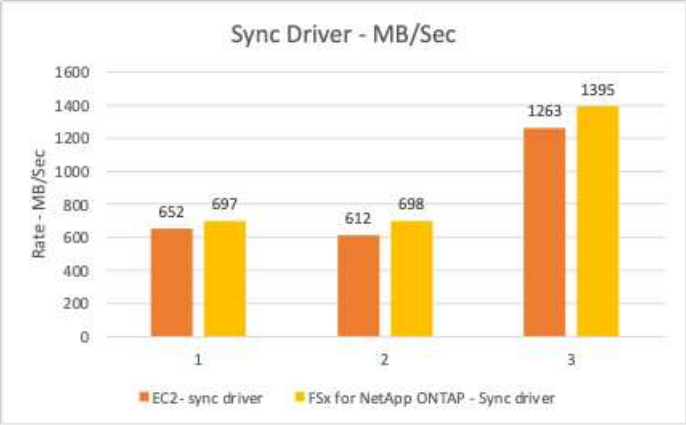
下表顯示 NetApp ONTAP 的 2GB/ 秒 FSX 和 Kafka 複寫係數 3 的 4GB/ 秒效能。複寫因素 3 會在 NetApp ONTAP 儲存設備的 FSX 上執行三次讀寫作業。處理量驅動程式的總速率是 881 MB/s 、在 NetApp ONTAP 檔案系統的 2GB/ 秒 FSX 上執行讀寫 Kafka 作業約 2.64 GB/ 秒、處理量驅動程式的總速率是 1328 MB/ 秒、可讀寫 Kafka 作業約 3.98 GB/ 秒。卡夫卡的效能是線性的、可根據 NetApp ONTAP 處理量的 FSX 進行擴充。





下表顯示 EC2 執行個體與 NetApp ONTAP 的 FSX 之間的效能（ Kafka 複寫係數： 3 ）

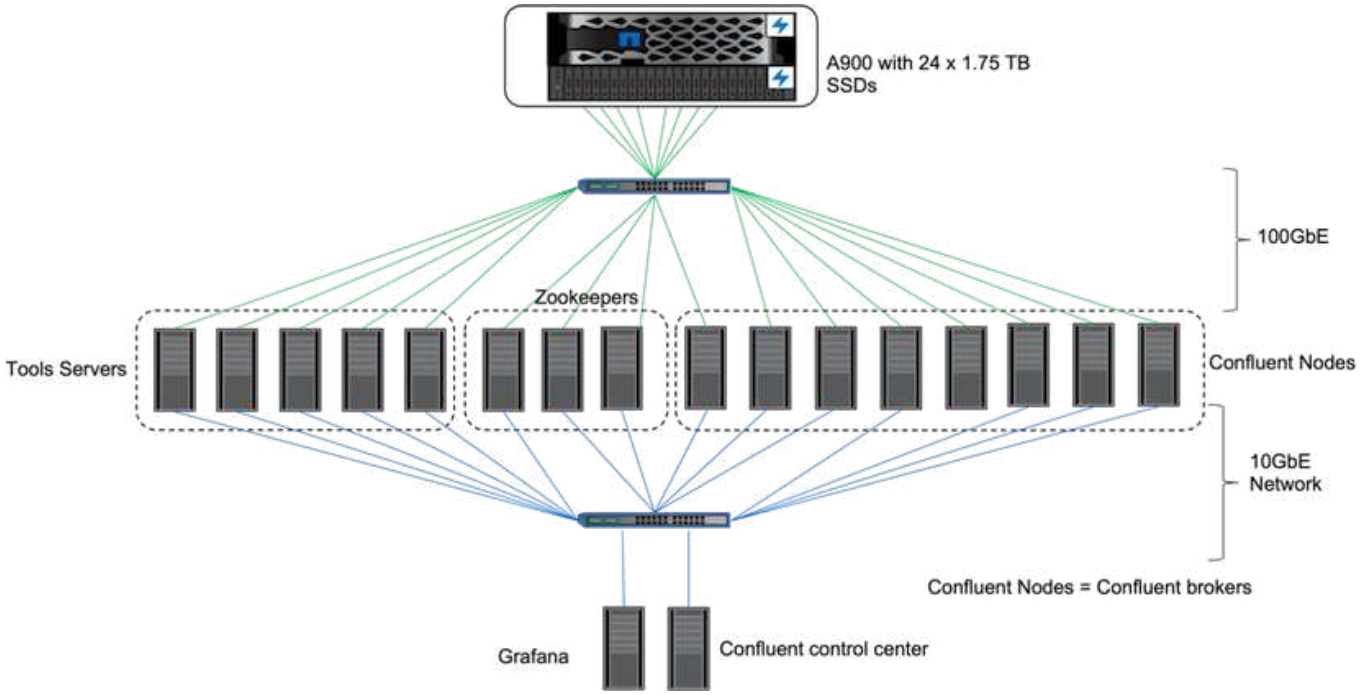




透過AFF 內部部署的VMware解決方案實現效能總覽與驗證

內部部署時、我們使用NetApp AFF VMware不支援ONTAP VMware的儲存控制器搭配VMware不支援套件（更新版本）來驗證Kafka叢集的效能與擴充性。我們使用的測試平台與ONTAP 先前採用的階層式儲存最佳實務做法相同、只要搭配使用即可AFF 。

我們使用Conflent Kafka 6.2.0來評估AFF 《The》。叢集具備八個代理節點和三個zookeeper節點。為了進行效能測試、我們使用了五個OMB工作節點。



儲存組態

我們使用NetApp FlexGroups執行個體為記錄目錄提供單一命名空間、簡化還原與組態。我們使用NFSv4.1和pNFS提供直接路徑存取記錄區段資料。

用戶端調校

每個用戶端都使用FlexGroup 下列命令來掛載這個執行個體。

```
mount -t nfs -o vers=4.1,nconnect=16 172.30.0.121:/kafka_vol01
/data/kafka_vol01
```

此外、我們也增加了 `max_session_slots`` 預設值 `64` 至 `180`。這與ONTAP 不符合的預設工作階段插槽限制。

Kafka代理商調校

為了將測試中系統的處理量最大化、我們大幅增加了特定關鍵執行緒集區的預設參數。我們建議針對大多數組態、遵循Conflent Kafka最佳實務做法。此調校可將出色的I/O與儲存設備的並行性最大化。這些參數可調整以符

合代理商的運算資源和儲存屬性。

```
num.io.threads=96
num.network.threads=96
background.threads=20
num.replica.alter.log.dirs.threads=40
num.replica.fetchers=20
queued.max.requests=2000
```

工作負載產生器測試方法

我們使用與雲端測試相同的OMB組態來測試處理量驅動程式和主題組態。

1. 使用可在一個叢集上配置的可執行個體AFF FlexGroup 。

```
---
- name: Set up kafka broker processes
  hosts: localhost
  vars:
    ntap_hostname: ‘hostname’
    ntap_username: 'user'
    ntap_password: 'password'
    size: 10
    size_unit: tb
    vserver: vs1
    state: present
    https: true
    export_policy: default
    volumes:
      - name: kafka_fg_vol01
        aggr: ["aggr1_a", "aggr2_a", “aggr1_b”, “aggr2_b”]
        path: /kafka_fg_vol01
  tasks:
    - name: Edit volumes
      netapp.ontap.na_ontap_volume:
        state: "{{ state }}"
        name: "{{ item.name }}"
        aggr_list: "{{ item.aggr }}"
        aggr_list_multiplier: 8
        size: "{{ size }}"
        size_unit: "{{ size_unit }}"
        vserver: "{{ vserver }}"
        snapshot_policy: none
        export_policy: default
        junction_path: "{{ item.path }}"
        qos_policy_group: none
        wait_for_completion: True
        hostname: "{{ ntap_hostname }}"
        username: "{{ ntap_username }}"
        password: "{{ ntap_password }}"
        https: "{{ https }}"
        validate_certs: false
      connection: local
      with_items: "{{ volumes }}"
```

2. 已在ONTAP SVM上啟用pNFS。

```
vserver modify -vserver vs1 -v4.1-pnfs enabled -tcp-max-xfer-size 262144
```

3. 此工作負載是透過處理量驅動程式觸發、其工作負載組態與Cloud Volumes ONTAP 執行動作時相同。請參閱「」一節[穩定狀態效能]」。工作負載使用3個複寫係數、表示NFS中保留了三個記錄區段複本。

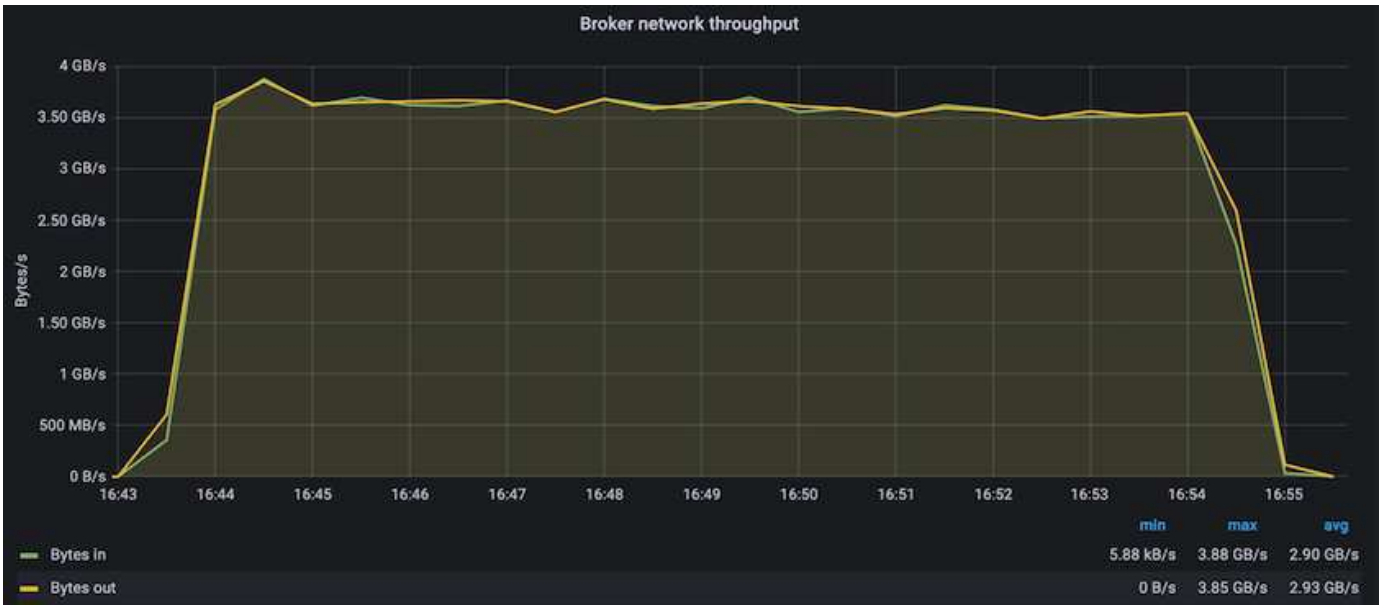
```
sudo bin/benchmark --drivers driver-kafka/kafka-throughput.yaml
workloads/1-topic-100-partitions-1kb.yaml
```

4. 最後、我們使用待處理項目來完成測量、以衡量消費者是否有能力掌握最新訊息。OMB會在測量開始時暫停使用者、藉此建構待處理項目。這會產生三個不同的階段：建立待處理項目（僅限生產商流量）、減少待處理項目（使用者負擔沉重的階段、消費者會在某個主題中趕上錯過的事件）、以及穩定狀態。請參閱「」一節探索儲存限制」以取得更多資訊。

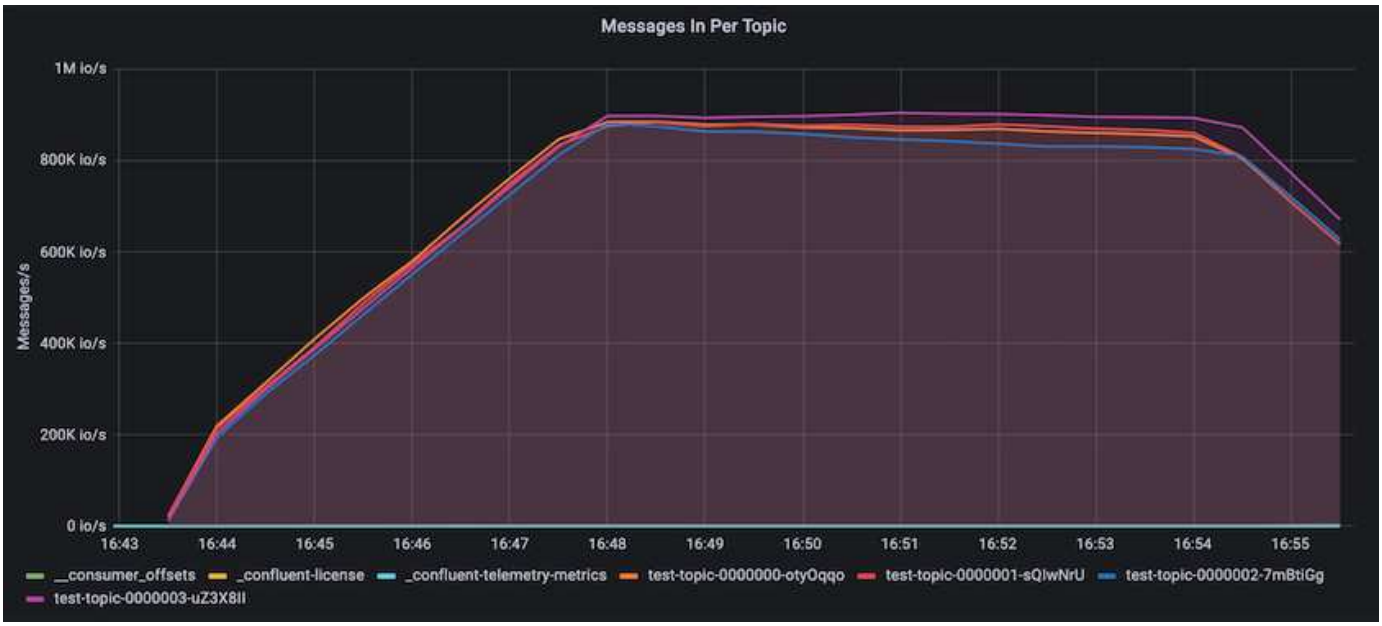
穩定狀態效能

我們使用AFF OpenMessaging基準測試評估了《支援不支援的資料」、以提供類似Cloud Volumes ONTAP 於AWS中的《支援的資料、支援的資料、以及AWS中的DAS。所有效能值均代表生產商與消費者層級的Kafka叢集處理量。

Conflent Kafka與AFF 《The》（《The》）的穩定狀態效能、使生產商與消費者的平均處理量均超過3.4GBps。這是卡夫卡叢集內超過340萬封訊息。透過將BrokerTopicMetrics的持續處理量以每秒位元組為單位視覺化、我們可以看到AFF 由VMware支援的卓越穩定狀態效能和流量。



這與每個主題所傳送訊息的檢視非常一致。下圖提供每個主題的詳細資料。在測試的組態中、我們在四個主題中、每個主題都看到將近900k個訊息。

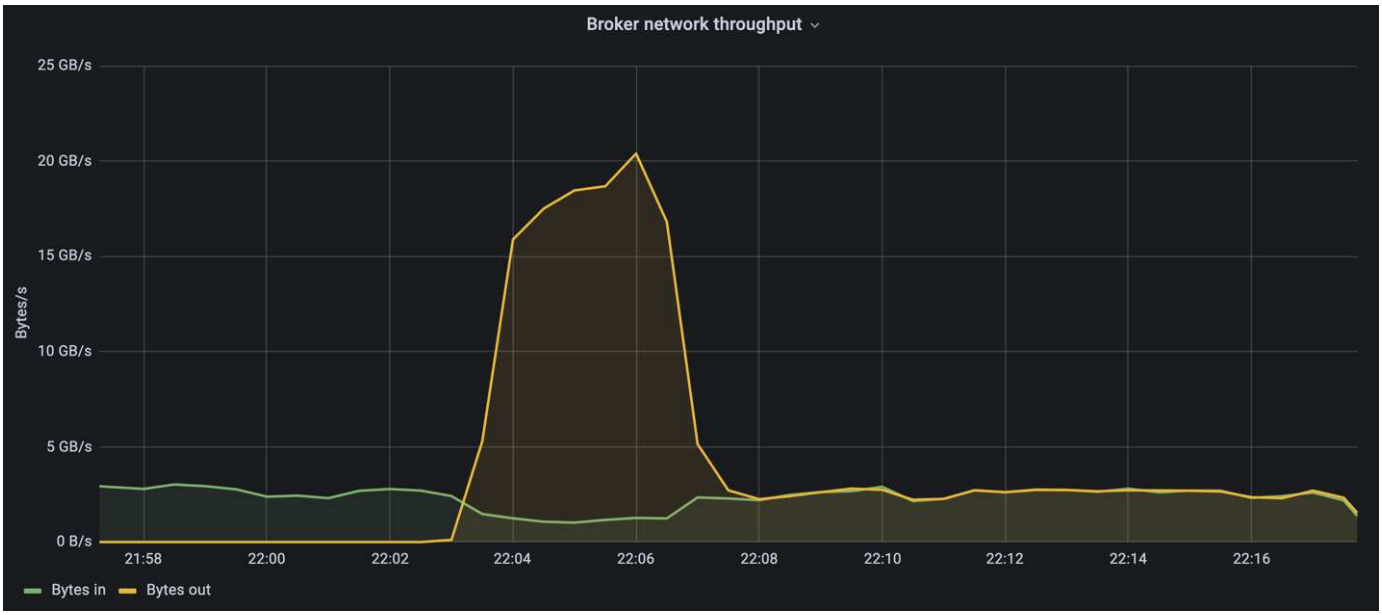


極致效能、探索儲存限制

針對這個解決方案、我們也使用待處理項目功能測試OMB AFF 。待處理項目功能會暫停訂閱消費者、而卡夫卡叢集中會建立大量的事件。在此階段中、只會發生產生者流量、產生提交至記錄的事件。這最能模擬批次處理或離線分析工作流程；在這些工作流程中、會啟動使用者訂閱、而且必須讀取已從代理快取中移出的歷史資料。

為了瞭解此組態中對使用者處理量的儲存限制、我們測量了純產生者階段、以瞭解A900可能會吸收多少寫入流量。請參閱下一節「[規模調整指南]」以瞭解如何運用這些資料。

在這項測量的純生產商部分期間、我們看到高尖峰處理量、使A900效能的極限推升（當其他代理商資源不飽和、無法為生產商和消費者流量提供服務時）。



我們將此測量的訊息大小增加至16k、以限制每個訊息的開銷、並將NFS掛載點的儲存處理量最大化。

```
messageSize: 16384
consumerBacklogSizeGB: 4096
```

Conflent Kafka叢集達到4.03GBps的尖峰生產量。

```
18:12:23.833 [main] INFO WorkloadGenerator - Pub rate 257759.2 msg/s /
4027.5 MB/s | Pub err     0.0 err/s …
```

在OMB填入事件待處理項目之後、使用者流量便會重新啟動。在測量待處理項目耗盡時、我們觀察到所有主題的尖峰使用者處理量都超過20Gbps。儲存OMB記錄資料的NFS磁碟區總處理量接近30Gbps。

規模調整指南

Amazon Web Services提供 "規模調整指南" 適用於Kafka叢集規模調整與擴充。

此規模提供了一種實用的公式、可用來判斷Kafka叢集的儲存處理量需求：

對於複寫係數為r的tcluster叢集所產生的彙總處理量、Broker儲存設備所接收的處理量如下：

```
t[storage] = t[cluster]/#brokers + t[cluster]/#brokers * (r-1)
           = t[cluster]/#brokers * r
```

這點可以進一步簡化：

```
max(t[cluster]) <= max(t[storage]) * #brokers/r
```

使用此公式可讓您針對ONTAP Kafka的熱階層需求、選擇適當的支援平台。

下表說明A900的預期生產商處理量、以及不同的複寫因素：

複寫因素	生產商處理量（**GPP**）
3（測量）	3.4.
2.	5.1
1.	10.2

結論

NetApp解決方案解決了這項不合理的重新命名問題、為先前與NFS不相容的工作負載、提供簡單、廉價且集中管理的儲存形式。

這項新模式可讓客戶建立更易於管理的Kafka叢集、以便更輕鬆地移轉和鏡射、以實現災難恢復和資料保護。我們也看到NFS提供更多優勢、例如降低CPU使用率、縮短恢復時間、大幅改善儲存效率、以及透過NetApp

ONTAP 功能提升效能。

何處可找到其他資訊

若要深入瞭解本文所述資訊、請檢閱下列文件和 / 或網站：

- 什麼是Apache Kafka？

  "https://www.confluent.io/what-is-apache-kafka/"

- 什麼是「愚蠢的重新命名」？

  "https://linux-nfs.org/wiki/index.php/Server-side_silly_rename"

- 串流應用程式讀取ONATP。

  "https://www.netapp.com/blog/ontap-ready-for-streaming-applications/"

- Kafka的rug-重新命名問題。

  "https://sbg.technology/2018/07/10/kafka-nfs/"

- NetApp產品文件

  "https://www.netapp.com/support-and-training/documentation/"

- 什麼是NFS？

  "https://en.wikipedia.org/wiki/Network_File_System"

- 什麼是Kafka分割區重新指派？

  "https://docs.cloudera.com/runtime/7.2.10/kafka-managing/topics/kafka-manage-cli-reassign-overview.html"

- 什麼是OpenMessaging基準測試？

  "https://openmessaging.cloud/"

- 如何移轉Kafka代理商？

  "https://medium.com/@sanchitbansal26/how-to-migrate-kafka-cluster-with-no-downtime-58c216129058"

- 您如何監控Prometheus的Kafka代理商？

  https://www.confluent.io/blog/monitor-kafka-clusters-with-prometheus-grafana-and-confluent/

- Apache Kafka的託管平台

  https://www.instaclustr.com/platform/managed-apache-kafka/

- 支援Apache Kafka

https://www.instaclustr.com/support-solutions/kafka-support/

- Apache Kafka諮詢服務

  https://www.instaclustr.com/services/consulting/

版權資訊

Copyright © 2024 NetApp, Inc. 版權所有。台灣印製。非經版權所有人事先書面同意，不得將本受版權保護文件的任何部分以任何形式或任何方法（圖形、電子或機械）重製，包括影印、錄影、錄音或儲存至電子檢索系統中。

由 NetApp 版權資料衍伸之軟體必須遵守下列授權和免責聲明：

此軟體以 NETAPP「原樣」提供，不含任何明示或暗示的擔保，包括但不限於有關適售性或特定目的適用性之擔保，特此聲明。於任何情況下，就任何已造成或基於任何理論上責任之直接性、間接性、附隨性、特殊性、懲罰性或衍生性損害（包括但不限於替代商品或服務之採購；使用、資料或利潤上的損失；或企業營運中斷），無論是在使用此軟體時以任何方式所產生的契約、嚴格責任或侵權行為（包括疏忽或其他）等方面，NetApp 概不負責，即使已被告知有前述損害存在之可能性亦然。

NetApp 保留隨時變更本文所述之任何產品的權利，恕不另行通知。NetApp 不承擔因使用本文所述之產品而產生的責任或義務，除非明確經過 NetApp 書面同意。使用或購買此產品並不會在依據任何專利權、商標權或任何其他 NetApp 智慧財產權的情況下轉讓授權。

本手冊所述之產品受到一項（含）以上的美國專利、國外專利或申請中專利所保障。

有限權利說明：政府機關的使用、複製或公開揭露須受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中的「技術資料權利 - 非商業項目」條款 (b)(3) 小段所述之限制。

此處所含屬於商業產品和 / 或商業服務（如 FAR 2.101 所定義）的資料均為 NetApp, Inc. 所有。根據本協議提供的所有 NetApp 技術資料和電腦軟體皆屬於商業性質，並且完全由私人出資開發。 美國政府對於該資料具有非專屬、非轉讓、非轉授權、全球性、有限且不可撤銷的使用權限，僅限於美國政府為傳輸此資料所訂合約所允許之範圍，並基於履行該合約之目的方可使用。除非本文另有規定，否則未經 NetApp Inc. 事前書面許可，不得逕行使用、揭露、重製、修改、履行或展示該資料。美國政府授予國防部之許可權利，僅適用於 DFARS 條款 252.227-7015(b)（2014 年 2 月）所述權利。

商標資訊

NETAPP、NETAPP 標誌及 http://www.netapp.com/TM 所列之標章均為 NetApp, Inc. 的商標。文中所涉及的所有其他公司或產品名稱，均為其各自所有者的商標，不得侵犯。